Table of Contents

1. - Airquality-Monitoring Dokumentation **5**
1.1. - Diese Dokumentation durchsuchen *5*
1.2. - Kürzlich aktualisierte Seiten *5*
1.3. - Motivation und Zielsetzung - CM **6**
1.3.1. - 1. Einleitung *6*
1.3.2. - 2. Ist-Zustand *6*
1.3.2.1. - 2.1 Abhängigkeit von Cloud-Diensten *6*
1.3.2.2. - 2.2 Eingeschränkte Automatisierung *6*
1.3.2.3. - 2.3 Fehlende kontextuelle Historie *6*
1.3.3. - 3. Zielsetzung *7*
1.3.3.1. - 3.1 Echtzeit-Erfassung und -Bewertung *7*
1.3.3.2. - 3.2 Integrierte Aktoren-Automatisierung *7*
1.3.3.3. - 3.3 Vollständig lokaler Betrieb *7*
1.3.3.4. - 3.4 Modulares Ökosystem über MQTT *7*
1.3.3.5. - 3.5 Session-basiertes Datenmanagement *7*
1.3.4. - 4. Nutzen und Alleinstellungsmerkmal *7*
1.4. - Gesamtübersicht / Architektur - CM **9**
1.4.1. - 1. Zusammenspiel der Komponenten *9*
1.4.2. - 2. Architekturdiagramm *9*
1.4.3. - 3. Erläuterung der Architektur *10*
1.5. - Backend **12**
1.5.1. - Raspberry Pi Einrichtung & Remote-Workflow - PP **13**
1.5.1.1. - 1.SSH-Zugang am Pi aktivieren *13*
1.5.1.2. - 2. IP-Adresse des Pi ermitteln *13*
1.5.1.3. - 3. VPN-Verbindung ins THWS-Netz für Bitbucket etc. *13*
1.5.1.4. - 4. Remote-Entwicklung mit VS Code *14*
1.5.1.4.1. - Zusammenfassung *14*
1.5.2. - API - SK **15**
1.5.2.1. - Überblick *15*
1.5.2.2. - Zentrale Methoden *15*
1.5.2.2.1. - Startup & Initialisierung *15*
1.5.2.2.2. - Zentraler Loop *15*
1.5.2.2.3. - Websocket *18*
1.5.2.3. - Weitere API Endpoints *19*
1.5.2.4. - API Server starten *19*
1.5.3. - Session-Management im Code - PP **21**
1.5.3.1. - 1. Session-Beginn und -Beendigung (API-Workflow) *21*
1.5.3.1.1. - Start einer Session *21*
1.5.3.1.2. - Beenden einer Session *21*
1.5.3.1.3. - Session-Fortsetzung und Duplikatvermeidung *21*
1.5.3.2. - 2. Vorteile der Session-Verwaltung *21*
1.5.3.3. - 3. Beispiele aus dem Code *22*
1.5.4. - Datenbank-Design & Implementierung - PP **23**
1.5.4.1. - 1. Was sind „Sessions“? – Konzept und Nutzen *23*
1.5.4.1.1. - Hintergrund und Motivation: *23*
1.5.4.1.2. - Eine Session umfasst immer: *23*
1.5.4.1.3. - Ablauf im System: *23*
1.5.4.1.4. - Vorteile des Session-Konzepts: *23*
1.5.4.2. - 2. Aufbau und Tabellenstruktur *23*
1.5.4.2.1. - sessions *23*
1.5.4.2.2. - air_data *24*
1.5.4.3. - 3. Beziehungen und Regeln *24*
1.5.4.4. - 4. Datenhaltung und Insert-Logik *25*
1.5.4.5. - 5. Fehlerbehandlung und Migration *25*

1.5.4.6. - 6. Visualisierung 25
1.5.5. - Device Steuerung durch WLAN-Steckdosen - SK 27
1.5.5.1. - Ziel 27
1.5.5.2. - Umsetzung 27
1.5.5.2.1. - Python Library 27
1.5.5.2.2. - tp100_controller.py (WLAN-Steckdosen Controller) 27
1.5.5.2.3. - air_control_rules.py (Regeln zur Steuerung) 27
1.5.5.2.3.1. - Ventilator 28
1.5.5.2.3.2. - Diffusor 28
1.5.6. - Testing & Qualitätssicherung - PP 29
1.5.6.1. - 1. Teststruktur und -aufbau 29
1.5.6.2. - 2. Testausführung 29
1.5.6.3. - 3. Besondere Testmethoden und Tools 29
1.5.6.4. - 4. Beispiel-Testfall (test_save_data.py) 29
1.5.6.5. - 5. Typische Fehlerbilder beim Testen und Lösungen 30
1.5.6.6. - 6. Aktualisieren der requirements.txt 30
1.5.7. - Air Quality Rating - SK 31
1.5.7.1. - Ziel 31
1.5.7.2. - Umsetzung 31
1.5.7.2.1. - Aufteilung in 5 Kategorien 31
1.5.7.2.2. - Konzentrieren auf 5 Messwerte. Die Auswahl basiert auf gesundheitsrelevanter Aussagekraft, Sensorverfügbarkeit und Praxisnähe 31
1.5.7.2.3. - Trennung von Indoor und Outdoor 31
1.5.7.2.4. - Auswählen der Schwellenwerte zum Einteilen in die obigen 5 Kategorien 31
1.5.7.2.5. - Gesamtwertung 32
1.5.7.2.5.1. - Alter Ansatz 32
1.5.7.2.5.2. - Verbesserter Ansatz 32
1.5.8. - Luftsensor auslesen - SK 34
1.5.8.1. - Sensorwerte und Bedeutung 34
1.5.8.2. - read_sensor_data.py 34
1.5.8.3. - SensorReader – Wichtige Codeausschnitte 34
1.5.8.3.1. - Initialisierung und Verbindung 35
1.5.8.3.2. - Einlesen roher Zeile 35
1.5.8.3.3. - Parsen in Dict 35
1.5.8.3.4. - Weitere bereitgestellte Methoden 35
1.5.8.3.5. - Kurzbeispiel der Verwendung 36
1.6. - Frontend - MK 37
1.6.1. - 1. Motivation 37
1.6.2. - 2. Gesamtübersicht (Frontend-intern) 37
1.6.2.1. - Technologie-Stack 37
1.6.2.2. - Frontend-Architektur 37
1.6.2.3. - Zusammenspiel 37
1.6.3. - 3. Wichtige Abläufe 37
1.6.3.1. - 3.1 Gerätesteuerung (Ventilator/Diffusor) 37
1.6.3.2. - 3.2 Session-Verwaltung 38
1.6.3.3. - 3.3 Live-Daten (falls genutzt) 38
1.6.4. - 4. Wichtige Codestellen 38
1.6.4.1. - 4.1 src/components/DeviceControlPanel.jsx 38
1.6.4.2. - 4.2 src/services/api.js 38
1.6.4.3. - 4.3 src/utils/exportSession.js 38
1.6.4.4. - 4.4 src/utils/envShim.js 38
1.6.5. - 5. Installation / Einrichtung 39
1.6.5.1. - ENV-Konfiguration 39
1.6.6. - 6. Tests & Qualitätssicherung 39
1.6.7. - 7. Architekturelle Entscheidungen (Frontend) 39
1.6.8. - 8. Betriebshinweise 39
1.6.9. - 9. Detailierte Frontend-Implementierung 39
1.6.9.1. - 9.1 Services (src/services/api.js) 40

1.6.9.2. - 9.2 Hooks . . . 40
1.6.9.2.1. - useLiveStream(active) . . . 40
1.6.9.2.2. - useSessionDelta(sessionId, enabled, pollingMs) . . . 40
1.6.9.3. - 9.3 Wichtige Komponenten . . . 40
1.6.9.3.1. - Dashboard.jsx . . . 40
1.6.9.3.2. - DeviceControlPanel.jsx . . . 40
1.6.9.3.3. - HistoryChart.jsx . . . 41
1.6.9.3.4. - AirQualitySummary.jsx . . . 41
1.6.9.3.5. - ParticleChartCard.jsx . . . 41
1.6.9.3.6. - SensorMetricCard.jsx . . . 41
1.6.9.4. - 9.4 Utilities . . . 41
1.6.9.5. - 9.5 Tests . . . 41
1.6.9.6. - 9.6 ENV & Mocking-Strategie . . . 41
1.6.9.7. - 9.7 Styling & UI-Konzept . . . 42
1.6.9.8. - 9.8 Router & Entry-Point . . . 42
1.6.10. - 10. ToDos / Weiterer Ausbau (Frontend) . . . 42
1.6.11. - 11. Screenshots/Visualisierungen des Frontends/Komponenten (Mock-Betrieb) . . . 42
1.7. - LED-Ampel & MQTT - CM . . . **45**
1.7.1. - Vorbereitung und Hardware - CM . . . **46**
1.7.1.1. - Das Hardware-Setup: . . . 46
1.7.1.2. - Weitere nötige vorbereitende Maßnahmen am Raspberry Pi (oder zum Test parallel am Laptop/PC): . . . 49
1.7.1.3. - MQTT-Setup Troubleshooting - CM . . . **51**
1.7.1.3.1. - Troubleshooting am Raspberry Pi: . . . 51
1.7.2. - PlatformIO als Build-Tool - CM . . . **52**
1.7.3. - Code für den ESP32 - CM . . . **53**
1.7.3.1. - Übersicht der Code-Komponenten - CM . . . **54**
1.7.3.2. - Wichtige Code-Bestandteile im Detail - CM . . . **55**
1.7.3.2.1. - config.h . . . 55
1.7.3.2.2. - FadeHelper . . . 55
1.7.3.2.3. - LedController . . . 55
1.7.3.2.4. - MqttManager . . . 58
1.7.3.2.5. - WiFiManager.cpp . . . 59
1.7.3.2.6. - main.cpp . . . 60
1.7.3.2.7. - platform.ini . . . 63
1.7.4. - Teststrategie und Testausführung - CM . . . **64**
1.7.4.1. - Ziel der Tests . . . 64
1.7.4.2. - Testansatz . . . 64
1.7.4.2.1. - 1. Logikfunktionen ohne Hardwarebezug . . . 64
1.7.4.2.2. - 2. Hardwareabhängige Komponenten . . . 64
1.7.4.3. - Konkrete Teststrategie . . . 64
1.7.4.4. - Begründung für diesen Ansatz . . . 64
1.7.4.5. - Ergänzende Testmaßnahmen . . . 65
1.7.4.6. - Beispielhafter Testablauf . . . 65
1.7.4.7. - Anleitung: Ausführung der Unit-Tests auf dem PC . . . 65
1.7.4.7.1. - Voraussetzungen . . . 65
1.7.4.7.2. - Schritte: . . . 65
1.7.5. - MQTT-Publishing vom Raspberry Pi - CM . . . **67**
1.7.5.1. - Ziel . . . 67
1.7.5.2. - Umsetzung . . . 67
1.7.5.2.1. - 1. Initialisierung des MQTT-Clients . . . 67
1.7.5.2.2. - 3. Automatische Wiederverbindung zum MQTT-Broker . . . 68
1.7.5.3. - Fazit . . . 68
1.8. - Sonstiges . . . **69**
1.8.1. - Versionsverwaltung & Projektstruktur - PP . . . **70**
1.8.1.1. - 1. Projekt-Repository und Git-Workflows . . . 70
1.8.1.2. - 2. Virtuelle Umgebung & Abhängigkeiten . . . 70
1.8.1.3. - 3. Projektstruktur . . . 70

1.8.2. - Das Team **74**
1.9. - Deployment **75**
1.9.1. - Containerisierung mit Docker Compose - PP **76**
1.9.1.1. - 1. Übersicht *76*
1.9.1.2. - 2. docker-compose.yml (finales Setup) *76*
1.9.1.3. - 3. Erläuterungen zu den wichtigsten Optionen *76*
1.9.1.4. - 4. Start und Nutzung *77*
1.9.1.5. - 5. Besonderheiten und Tipps *77*
1.9.2. - Dockerfile Backend - CM **78**
1.9.2.1. - Zweck und Kontext *78*
1.9.2.2. - Dockerfile-Inhalt *78*
1.9.2.3. - Erläuterung der einzelnen Anweisungen *78*
1.9.2.4. - Hinweise zur Wartbarkeit *78*
1.9.2.5. - Zusammenfassung *79*
1.9.3. - Dockerfile Frontend - CM **80**
1.9.3.1. - Zweck und Kontext *80*
1.9.3.2. - Dockerfile-Inhalt *80*
1.9.3.3. - Erläuterung der einzelnen Anweisungen *80*
1.9.3.4. - Hinweise zur Wartbarkeit *80*
1.9.3.5. - Zusammenfassung *81*
1.10. - Abschluss und Erweiterungspotenzial - PP **82**
1.10.1. - Fazit *82*
1.10.2. - Erweiterungspotenzial *82*
1.10.3. - Lessons Learned *83*

Airquality-Monitoring Dokumentation

Alles erweitern Alles ausblenden

Diese Dokumentation durchsuchen

Kürzlich aktualisierte Seiten

- API - SK
  Juli 23, 2025 • aktualisiert von Krutzsch, Steffen • Änderung anzeigen
- Device Steuerung durch WLAN-Steckdosen - SK
  Juli 23, 2025 • aktualisiert von Krutzsch, Steffen • Änderung anzeigen
- Datenbank-Design & Implementierung - PP
  Juli 23, 2025 • aktualisiert von Putz, Pascal • Änderung anzeigen
- Session-Management im Code - PP
  Juli 23, 2025 • aktualisiert von Putz, Pascal • Änderung anzeigen
- Testing & Qualitätssicherung - PP
  Juli 23, 2025 • aktualisiert von Putz, Pascal • Änderung anzeigen

Mehr anzeigen

Motivation und Zielsetzung - CM

1. Einleitung

Die Bedeutung einer zuverlässigen Luftqualitätsüberwachung ist in Zeiten steigender Umweltbelastungen, wachsender Sensibilität für Gesundheitsthemen und verschärfter gesetzlicher Vorgaben kaum zu überschätzen. Feinstaub, flüchtige organische Verbindungen und andere Schadstoffe beeinträchtigen nicht nur das Wohlbefinden, sondern können langfristig zu ernsthaften Atemwegserkrankungen, Allergien oder kardiovaskulären Problemen führen. Gerade in Innenräumen, in denen sich Menschen oftmals über Stunden aufhalten, wirkt sich eine unzureichende Luftqualität unmittelbar auf Konzentration, Leistungsfähigkeit und allgemeines Wohlbefinden aus. Gleichzeitig nimmt der Trend zu Smart-Home-Systemen rasant zu: Automatisierte Regelkreise versprechen Komfort und Effizienz, doch werden die dafür benötigten Daten häufig in proprietären Cloud-Diensten verarbeitet – mit Abhängigkeiten von fremden Anbietern und potenziellen Datenschutzrisiken.

Vor diesem Hintergrund setzt das Projekt „Airquality-Monitoring“ an: Es kombiniert eine präzise Echtzeit-Sensorik mit einer vollständig lokal gehosteten Architektur und einem offenen, erweiterbaren Konzept. Durch die Integration einer fein aufgelösten Partikelmessung bis hinunter zu P0.1-Größen, die automatisierte Ansteuerung von Aktoren und ein flexibles, sessionbasiertes Daten-management entsteht eine Plattform, die nicht nur verlässliche Messergebnisse liefert, sondern sich gleichzeitig nahtlos in bestehende Home-Automation-Umgebungen einfügt – und das ganz ohne Cloud-Abhängigkeit. Dieses Bündel an Funktionalitäten adressiert aktuelle Herausforderungen im Bereich Raumluftqualität und schafft die Grundlage für eine skalierbare, datenschutzkonforme Lösung, die sowohl in privaten Haushalten als auch in öffentlichen Einrichtungen und Unternehmen eingesetzt werden kann.

---

2. Ist-Zustand

2.1 Abhängigkeit von Cloud-Diensten

- Viele Luftqualitäts-systeme nutzen proprietäre Plattformen, auf denen Datenerfassung, Verarbeitung und Visualisierung ausschließlich in der Cloud stattfinden.
- Nutzer sind oft gebunden an externe Accounts, Gebührenmodelle und eingeschränkte Datenexport-Schnittstellen.

2.2 Eingeschränkte Automatisierung

- Nur wenige Systeme bieten eine vollständig automatisierte Steuerung lokaler Aktoren.
- Wo Automatisierung existiert, erfolgt sie meist über externe APIs oder proprietäre Smart-Home-Hubs mit begrenzter Kompatibilität.

2.3 Fehlende kontextuelle Historie

- Historische Messwerte werden oft nur als einfache Zeitreihen dargestellt, ohne Möglichkeit zur separaten Verwaltung einzelner Mess-Kontexte (beispielsweise verschiedene Räume oder Mess-Sessions).

- Eine strukturierte Nachverfolgung und Auswertung nach zeitlichem oder räumlichem Kontext ist dadurch nur mit hohem manuellem Aufwand möglich.

---

3. Zielsetzung

3.1 Echtzeit-Erfassung und -Bewertung

- Kontinuierliche Auswertung sämtlicher relevanter Luftqualitätswerte (s. Luftsensor)
- Einordnung in fünf Qualitätsstufen (Sehr schlecht → Sehr gut) basierend auf international anerkannten Normen, Richtlinien und wissenschaftlichen Empfehlungen.

3.2 Integrierte Aktoren-Automatisierung

- Definition flexibler Regelwerke: je nach ermittelter Qualitätsstufe werden Smart-Plugs (Ventilator, Luftbefeuchter) automatisch via WiFi geschaltet
- Kombinierter Automatik- und manueller Modus im Dashboard, um unmittelbare Eingriffe zu ermöglichen

3.3 Vollständig lokaler Betrieb

- Backend und Frontend laufen in Docker-Containern auf einem einzigen Raspberry Pi – komplett ohne Cloud-Anbindung
- Datenspeicherung und -verarbeitung verbleiben vollständig in der lokalen Infrastruktur, was Ausfallrisiken minimiert und Datenschutz maximiert

3.4 Modulares Ökosystem über MQTT

- Veröffentlichung der aktuellen Qualitätsstufe als MQTT-Topic in Echtzeit
- Beliebige zusätzliche Komponenten benötigen lediglich ein MQTT-Abonnement, um direkt auf die Daten zugreifen zu können, ohne Backend-Anpassungen

3.5 Session-basiertes Datenmanagement

- Anlegen beliebiger Mess-Sessions mit frei wählbarem Namen und Ort
- Automatische Timestamp-Markierung und Persistenz aller Sensordaten in separaten Datenbanktabellen pro Session
- Einfache Auswahl und Analyse historischer Sessions über das Frontend-Dashboard

---

4. Nutzen und Alleinstellungsmerkmal

Lokal und Cloud-frei (Privacy-by-Design)	– Keine Abhängigkeit von externen Diensten – Vollständige Datenhoheit und Stabilität auch bei Internet-ausfall

	- Ideal für Behörden, Schulen und Unternehmen, die keine Daten in die Cloud geben dürfen
Automatisierte Aktorensteuerung	– Unabhängige Regelwerke direkt im Backend konfigurierbar – Direkte Rückkopplung zwischen Messung und Lüftungshardware
MQTT-basiertes Erweiterungskonzept	– Plug-and-Play: Neue Visualisierungspanels oder Alarmgeräte einfach per MQTT-Subscribe integrieren – Zukunftssichere Skalierung
Session-orientierte Historie	– Zielgerichtete Auswertung einzelner Situationen (z. B. Wohnräume, Büroräume, Außenmessungen) – Nur ein Sensor notwendig
Open Source & Community-Förderung	– Transparenter Code für private und gewerbliche Nutzung – Einfache Anpassung und Weiterentwicklung durch Dritte

1. **Maximale Kontrolle und Datenschutz**
   Da alle Komponenten lokal betrieben werden, verbleiben sämtliche Messdaten im eigenen Netzwerk. Dies erfüllt höchste Datenschutzanforderungen und eliminiert Risiken durch fremde Cloud-Provider.
2. **Einfache Erweiterbarkeit ohne Backend-Eingriffe**
   Neue Geräte oder Visualisierungsmedien können jederzeit hinzugefügt werden, indem sie das MQTT-Topic abonnieren. Das Backend bleibt unverändert – ideal für experimentelle Sensoren oder Alarmanlagen.
3. **Flexibles Session-Konzept**
   Eine innovative Gruppierung von Messwerten nach Sessions ermöglicht gezielte Analysen einzelner Messkampagnen: von Raum-zu-Raum-Vergleichen bis hin zu Außeneinsätzen.
4. **Automatisierungsfunktion als Kernfeature**
   Durch vordefinierte Regelwerke zur Steuerung von Ventilator und Luftbefeuchter wird Verschlechterungen der Luftqualität kontinuierlich und ohne nötiges manuelles Eingreifen des Nutzers aktiv entgegengewirkt.

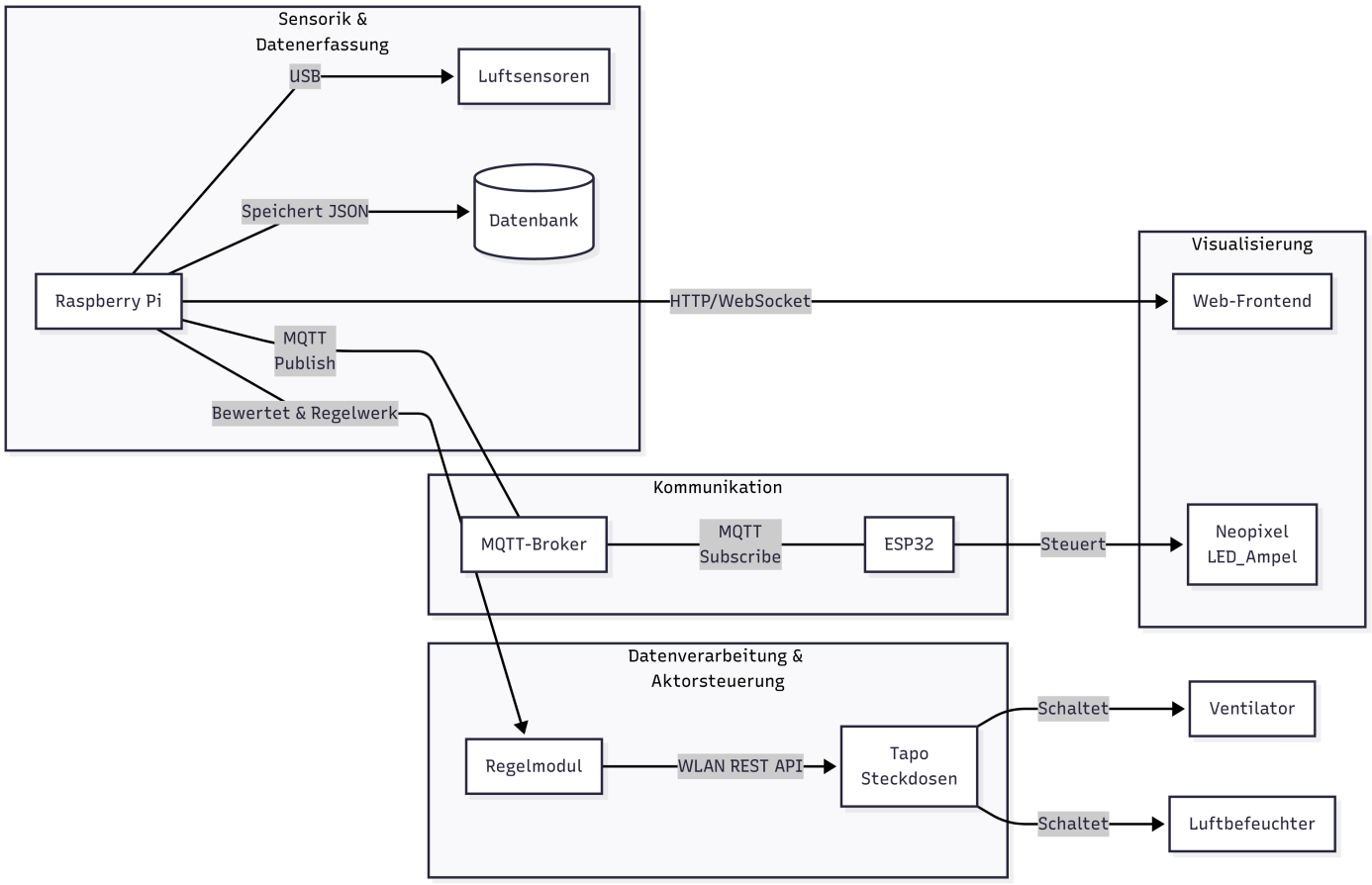
Gesamtübersicht / Architektur - CM

1. Zusammenspiel der Komponenten

Das Airquality-Monitoring-System besteht aus mehreren modularen Bausteinen, die gemeinsam die Erfassung, Bewertung, Steuerung und Visualisierung der Luftqualität in Echtzeit ermöglichen:

1. **Sensorik & Datenerfassung (Raspberry Pi + Canaree I5)**
   - Ein Raspberry Pi übernimmt über USB die Anbindung des Canaree I5-Sensors und liest Partikel- (bis P0.1), CO-, Temperatur- und Feuchtigkeitswerte ein.
   - Ein Python-Dienst (`SensorReader`) wandelt die Rohdaten zeilenweise in strukturierte Messobjekte um.
2. **Datenpersistenz & Session-Management**
   - Jede Messung wird mit Zeitstempel und – sofern aktiv – der aktuellen Session-ID in einer lokalen SQLite-Datenbank abgelegt.
   - Sessions lassen sich per API starten, benennen (Ort, Typ indoor/outdoor) und beenden, wodurch Messreihen klar getrennt und später gezielt ausgewertet werden können.
3. **Bewertung & Regelwerk**
   - Ein zentrales Modul (`AirQualityRules`) ordnet eingehende Messwerte kontinuierlich einer von fünf Luftgüteklassen zu (Sehr gut bis Sehr schlecht).
   - Parallel bestimmt das Regelmodul (`AirControlRules`), ob gemäß den aktuellen Grenzwerten Ventilator oder Luftbefeuchter angeschaltet bzw. abgeschaltet werden müssen.
4. **Aktorsteuerung (Tapo-Steckdosen)**
   - Über das Modul `TP100Controller` sendet das Backend per WLAN-REST an zwei Tapo-Smart-Plugs:
     - **Ventilator-Steckdose**
     - **Luftbefeuchter-Steckdose**
   - Neben dem automatischen Modus erlauben die REST-Endpoints auch manuelle „on/off/auto“-Befehle, die das Frontend bereitstellt.
5. **MQTT-Kommunikation & externe Displays**
   - Das Backend veröffentlicht die aktuell errechnete Gesamtgüteklasse per MQTT-Client nonstop auf dem Topic `airquality/command/led`.
   - Ein MQTT-Broker (z. B. Mosquitto) vermittelt die Nachrichten an beliebige Abonnenten.
   - Der selbstgebaute ESP32-DevKit mit Adafruit-NeoPixel-Ring übernimmt als Beispiel-Subscriber die visuelle Ampelanzeige der Luftgüte (Farben und Blinkmuster).
6. **Web-Frontend & Live-Dashboard**
   - Ein React-basiertes Dashboard verbindet sich via HTTP/WebSocket mit dem FastAPI-Backend.
   - Es zeigt Live-Messwerte und Güteklassen, erlaubt Session-Start/Ende, listet historische Sessions auf und visualisiert deren Daten in Graphen.
   - Über Schalter im UI lassen sich beide Steckdosen in den manuellen Modus überführen und direkt schalten.

2. Architekturdiagramm



3. Erläuterung der Architektur

1. **Startup & Initialisierung**
   Beim Start lädt FastAPI das Datenbankschema, initialisiert den Sensor-leser und verbindet den MQTT-Client. Zwei Hintergrund-Tasks starten:
   - **`live_data_loop`** (Sekunde für Sekunde Messung, Bewertung, Persistenz, Aktor-Steuerung, MQTT-Publish, WebSocket-Broadcast)
   - **`mqtt_reconnect_loop`** (sichert automatische Wiederverbindung zum Broker)
2. **Sensordaten → Datenbank & Rating**
   - Jede empfangene Rohzeile wird geparst und – falls eine Session aktiv ist – in der Tabelle `air_data`gespeichert.
   - `AirQualityRules` berechnet aus den fünf Schlüsselmesswerten die Gesamtgüte (String + Farbe).
3. **Automatische Aktorsteuerung**
   - Basierend auf `AirControlRules` prüft das System, ob Ventilator bzw. Diffusor ein- bzw. ausgeschaltet werden müssen.
   - Die REST-Aufrufe an `TP100Controller` schalten die Tapo-Steckdosen entsprechend.
4. **Datenverteilung**
   - **MQTT:** Gesamtgüte als String wird im Topic `airquality/command/led` veröffentlicht.
   - **WebSocket:** Ein JSON-Payload mit Timestamp, Messwerten, Einzel-Ratings, Gesamtstatus, Farben und Geräte-zuständen geht an alle verbundenen Frontend-Clients.
5. **Web-Frontend**
   - Abfragen aller Sessions (`/sessions`), Starten/Beenden neuer Sessions (`/sessions`, `/sessions/{id}/end`)

- Live-Anzeige über `/ws/live-data`, manuelle Steuerung via `/devices/ventilator/control` und `/devices/diffusor/control`
- Historische Datenabrufe über `/sessions/{id}/data`

Durch diese modulare, event- und regelbasierte Architektur entsteht ein flexibles System, das sowohl Batch-Analysen (Sessions) als auch hochfrequentes Echtzeit-Monitoring und -Steuern vereint – alles lokal und einfach erweiterbar.

Backend

Raspberry Pi Einrichtung & Remote-Workflow - PP

1.SSH-Zugang am Pi aktivieren

Zuerst wurde SSH auf dem Pi aktiviert, um eine sichere Fernsteuerung über das Netzwerk zu ermöglichen:

1. **Raspi-Config öffnen:**
   Am Pi einloggen (Monitor + Tastatur) und im Terminal ausführen:
   `sudo raspi-config`
2. **SSH aktivieren:**
   Im Menü zu „Interfacing Options“ > „SSH“ navigieren und auf „Enable“ stellen.
3. **SSH-Dienst läuft:**
   Ab jetzt akzeptiert der Pi SSH-Verbindungen automatisch nach jedem Systemstart.

---

2. IP-Adresse des Pi ermitteln

Um den Pi über das Netzwerk anzusprechen, wurde die aktuelle lokale IP-Adresse wie folgt ausgelesen:
`hostname -I`

Beispiel-Ausgabe:
`192.168.178.68`

Die Adresse wurde für alle späteren SSH- und VS Code-Remote-Verbindungen benötigt.

---

3. VPN-Verbindung ins THWS-Netz für Bitbucket etc.

Um aus dem externen Netzwerk auf interne Dienste (wie das THWS-Bitbucket-Repo) zugreifen zu können, wurde der VPN-Client **openfortivpn** eingerichtet:

1. **Installation:**
   `sudo apt update sudo apt install -y openfortivpn`
2. **Konfigurationsdatei anlegen:**
   `sudo nano /etc/openfortivpn/config`

   Mit folgendem (angepassten) Inhalt:

   ```
   host = <vpn.thws.de>
   port = 443
   username = <thws_benutzername>
   password = <thws_passwort>
   ```

3. **VPN-Verbindung herstellen:**
   `sudo openfortivpn`

   → Die Verbindung bleibt solange aktiv, wie das Terminal offen bleibt.

   Beispielausgabe:
   ```
   FortiGate VPN SSL – Version x.x.x
   Virtual IP: 10.x.x.x
   ```

---

4. Remote-Entwicklung mit VS Code

Für effiziente Entwicklung und Deployment wurde **Visual Studio Code** mit dem Plugin „Remote – SSH“ genutzt:

- **Remote-SSH-Extension installieren**
- Unten links das SSH-Symbol klicken → „Connect to Host…“
- SSH-Adresse eingeben (z. B. `airquality@192.168.178.68`)
- Nach Passwortabfrage verbindet sich VS Code direkt auf das Dateisystem und Terminal des Pi
- So kann direkt auf dem Pi programmiert, getestet und deployed werden

---

Zusammenfassung

- SSH und VPN erlauben vollwertiges Remote-Development am Pi
- VS Code bietet direkten Zugriff auf Dateien, Terminal und Tools (Python, pip, git, etc.)
- **Alle Entwicklungs- und Deploymentschritte werden so komfortabel aus der Ferne durchgeführt**

API - SK

Überblick

`api.py` ist das zentrale Backend des AirQualityMonitorProjekts und bildet die REST/WebSocketSchnittstelle für das Frontend. Es basiert auf **FastAPI** und kümmert sich um

- Continuous LiveDatenStreaming vom Luftsensor
- Persistenz über SQLite
- Gerätesteuerung (Ventilator & AromaDiffusor)
- MQTTWeiterleitung des LuftqualitätsRatings

FastAPI wurde gegenüber Flask gewählt, da es eine bessere native Unterstützung für WebSockets bietet, was für die zuverlässige Echtzeitübertragung der Sensordaten entscheidend ist.

Zentrale Methoden

Startup & Initialisierung

- Initialisiert Datenbank, Sensor, MQTTClient und holt sich den aktuellen Status der WLAN-Steckdosen.
- Startet die beiden HintergrundTasks `live_data_loop()` und `mqtt_reconnect_loop()`.

```
@app.on_event("startup")
async def startup_event():
    global sensor_reader, mqtt_client
    conn = init_db()
    conn.close()

    try:
        sensor_reader = SensorReader()
        print("Sensor erfolgreich initialisiert")
    except Exception as e:
        print(f"Sensor konnte nicht initialisiert werden: {e}")
        sensor_reader = None

 try:
        ventilator = TP100Controller(VENTILATOR_IP, TAPO_EMAIL, TAPO_PASSWORD)
        diffusor = TP100Controller(DIFFUSOR_IP, TAPO_EMAIL, TAPO_PASSWORD)
        last_vent_state = ventilator.get_status()
        last_diff_state = diffusor.get_status()
    except Exception as e:
        print(f"Fehler beim Abrufen des Gerätestatus beim Start: {e}")

    try:
        mqtt_client = MQTTClient(MQTT_BROKER, MQTT_PORT, MQTT_TOPIC)
        print(f"MQTT Client verbunden mit {MQTT_BROKER}:{MQTT_PORT}, Topic: {MQTT_TOPIC}")
    except Exception as e:
        print(f"MQTT Client konnte nicht verbunden werden: {e}")
        mqtt_client = None

    asyncio.create_task(live_data_loop())
    asyncio.create_task(mqtt_reconnect_loop())
```

Zentraler Loop

Die Funktion `live_data_loop()` bildet das Herz des Systems: Er sorgt dafür, dass Messdaten im Sekundentakt zu aussagekräftigen Informationen werden, bei Bedarf Geräte ansteuert und alle Ergebnisse sofort an UI & IoT verteilt.

Schritt	Hauptverantwortung	Was passiert
1 Erfassen	Sensordaten einsammeln	Jede Sekunde liest `sensor_reader.lese_daten()` eine kommagetrennte Key-Value Liste ("Rohzeile").
2 Umwandeln	Rohzeile → PythonDict	`sensor_reader.parse_sensor_line(raw_line)`transformiert die eingelesene Rohzeile in ein Python-Dict.
3 Persistieren	In Datenbank schreiben	Läuft eine Session, speichert `insert_data()` die Rohzeile + Timestamp in SQLite.
4 Extrahieren	Bewertungsmesswerte extrahieren	`sensor_reader.get_rating_measurements(raw_line)` filtert die fünf Parameter, die für die LuftqualitätsRegeln relevant sind (PM2.5, CO2eq, VOCAQI, TEMP, HUMID).
5 Bewerten	Luftqualitätratings berechnen	`AirQualityRules` vergleicht Messwerte mit Schwellen und erzeugt ein GesamtRating und Ratings der Bewertungsmesswerte („SEHR GUT “ … „SEHR SCHLECHT“).
6 Veröffentlichen	Ampel mit MQTT steuern	Das GesamtRating wird an ein MQTTTopic gepublisht und steuert Ampel.
7 Steuerung	Geräte automatisch steuern	`AirControlRules` entscheidet anhand des Ratings, ob Ventilator und AromaDiffusor per TP100Steckdose ein/aus-geschaltet werden, falls der automatische Modus aktiviert ist.
8 Aufbereiten & Verteilen	Daten für Frontend aufbereiten und broadcasten	Ein JSONPayload (Messwerte, Ratings, Farben, AktorStatus …) wird an alle WebSockets gebroadcastet.
9 Takten & Resilienz	Loop stabil halten	`await asyncio.sleep(1)` stellt 1 Hz sicher. Ein globales `try/except` fängt Fehler ab; nach kurzer Wartezeit (5 s) startet der nächste Zyklus – der Dienst bleibt dauerhaft online.

Bemerkung zur Bewertung:

Bei `AirQualityRules` kann man zwischen Indoor und Outdoor Regeln entscheiden. In `api.py` wird standardmäßig die Indoor Regeln verwendet, da dies der typische Anwendungsfall ist. Wenn eine Outdoor Session aktiv ist, werden hingegen die Outdoor Regeln verwendet. `get_air_quality_rules()` holt sich dabei von der aktuellen Session den Typ (Indoor/Outdoor) und dementsprechend gibt es die passenden Regeln von AirQualityRules zurück.

```
async def live_data_loop():
    global current_session_id, mqtt_client
    global ventilator_auto, diffusor_auto, last_vent_state, last_diff_state

    while True:
        try:
            if not sensor_reader:
                await asyncio.sleep(1)
                continue

            raw_line = sensor_reader.lese_daten()
            if not raw_line:
```

```
    await asyncio.sleep(1)
    continue

# Vollständige Parser-Daten holen
parsed = sensor_reader.parse_sensor_line(raw_line)

# Wenn Session aktiv, Raw-Line in DB speichern
if current_session_id:
    conn = get_db_connection()
    from database.save_data import insert_data
    insert_data(conn, raw_line, current_session_id)
    conn.close()

# Nur die fünf Schlüsselmesswerte für die Rating-Logik extrahieren
rating_vals = sensor_reader.get_rating_measurements(raw_line)

# Gesamt- und Einzel-Rating berechnen
rules = get_air_quality_rules()
overall_quality = rules.get_overall_air_quality(rating_vals)
overall_quality_str = air_quality_to_string(overall_quality)

# MQTT Publish
if mqtt_client:
    try:
        mqtt_client.publish(overall_quality_str)
    except Exception as e:
        print(f"Fehler beim MQTT Publish: {e} - MQTT Client wird neu verbunden")
        mqtt_client = None # löst den reconnect im mqtt_reconnect_loop() aus

# Automatische Steuerung
control_rules = get_air_control_rules()
if ventilator_auto:
    global last_vent_state
    if control_rules.needs_ventilation(rating_vals) and last_vent_state != "on":
        TP100Controller(VENTILATOR_IP, TAPO_EMAIL, TAPO_PASSWORD).turn_on()
        last_vent_state = "on"
    elif control_rules.ventilation_not_needed(rating_vals) and last_vent_state != "of
        TP100Controller(VENTILATOR_IP, TAPO_EMAIL, TAPO_PASSWORD).turn_off()
        last_vent_state = "off"

if diffusor_auto:
    global last_diff_state
    if control_rules.diffusor_should_be_on(rating_vals) and last_diff_state != "on":
        TP100Controller(DIFFUSOR_IP, TAPO_EMAIL, TAPO_PASSWORD).turn_on()
        last_diff_state = "on"
    elif control_rules.diffusor_should_be_off(rating_vals) and last_diff_state != "of
        TP100Controller(DIFFUSOR_IP, TAPO_EMAIL, TAPO_PASSWORD).turn_off()
        last_diff_state = "off"

print(
    "[DEBUG] Steckdosen  |  "
    f"Ventilator: {'AUTO' if ventilator_auto else 'MAN'} / {last_vent_state or 'unkno
    f"Diffusor: {'AUTO' if diffusor_auto else 'MAN'} / {last_diff_state or 'unknown'}
)

ratings = {}
for param, val in rating_vals.items():
    rating = rules.rate_parameter(val, param)
    ratings[param] = {
        'value': val,
        'rating': air_quality_to_string(rating),
        'color': get_air_quality_colors()[air_quality_to_string(rating)]
    }

measurements_all = {
    k: v for k, v in parsed.items()
    if isinstance(v, (int, float))
}

# Payload bauen und broadcasten
```

```
            live_data = {
                'timestamp': datetime.now().isoformat(),
                'measurements': measurements_all,
                'air_quality_rating': ratings,
                'overall_quality': overall_quality_str,
                'overall_color': get_air_quality_colors()[overall_quality_str],
                'parsed_data': parsed,
                'session_type': get_current_session_type().value,
                'ventilator_auto':  ventilator_auto,
                'diffusor_auto':    diffusor_auto,
                'ventilator_state': last_vent_state,
                'diffusor_state':   last_diff_state
            }
            await broadcast_live_data(live_data)

            await asyncio.sleep(1)

        except Exception as e:
            print(f"Error in live data loop: {e}")
            await asyncio.sleep(5)
```

Bemerkung zu TP100Controller:

Das `TP100Controller`-Objekt wird gezielt nur dann erzeugt, wenn eine Steueraktion tatsächlich erforderlich ist. Wir verzichten bewusst auf globale Instanzen, da eine Verbindung zur Steckdose nur im Moment der Steuerung benötigt wird. So bleiben Verbindungsfehler auf einzelne Steuerbefehle begrenzt, ohne das Gesamtsystem zu beeinträchtigen.

Websocket

WebSocket `/ws/live-data`

- Der Client baut einmalig eine WSVerbindung zum Server auf.
- `await websocket.accept()` bestätigt den Handshake.
- Der verbundene Socket wird in die globale Liste `websocket_connections` aufgenommen.
- So behält das Backend alle offenen Kanäle im Blick.
- Danach wartet die Coroutine mit `await websocket.receive_text()` auf eingehende Nachrichten.
- *Der Server erwartet aber nichts – der Aufruf blockiert lediglich, bis der Client die Verbindung schließt.*
- Ergebnis: PushOnlyKanal ohne PingPongLogik.
- Im `finally`Block wird der Socket wieder aus der Liste entfernt, egal ob der Client sauber schließt oder die Verbindung
- abreißt.
- Damit verhindert man Speicher/Listenleichen.

```
@app.websocket("/ws/live-data")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    websocket_connections.append(websocket)
    try:
        await websocket.receive_text()
    finally:
        if websocket in websocket_connections:
            websocket_connections.remove(websocket)
```

`broadcast_live_data(data)`

- Wird jede Sekunde vom LiveDataLoop aufgerufen.
- Liefert ein JSONSerialisat des MessSnapshots an alle aktuell verbundenen Clients.

- Sofortiger Exit, wenn niemand verbunden ist spart CPU.
  Die Liste wird vor dem Iterieren kopiert (`connections_copy`).
- Grund: Parallel könnten andere Koroutinen denselben Pool verändern (Clients verbinden/trennen).
  Die Kopie schützt vor „size changed during iteration“.
  FehlerHandling pro Socket:
- `try … except` fängt z. B. „Broken pipe“ ab, loggt den Fehler und entfernt nur den betroffenen Socket.
  So bleibt der Broadcast für alle übrigen Clients intakt.
- Serialisierung mit `json.dumps(data)` unmittelbar vor dem Senden; Clients erhalten reine Textframes.

```
async def broadcast_live_data(data: Dict[str, Any]):
    if not websocket_connections:
        return

    connections_copy = websocket_connections.copy()

    for websocket in connections_copy:
        try:
            await websocket.send_text(json.dumps(data))
        except Exception as e:
            print(f"Error sending to WebSocket: {e}")
            if websocket in websocket_connections:
                websocket_connections.remove(websocket)
```

Weitere API Endpoints

Im folgenden ein Überblick über die API Endpoints, die neben dem Websocket noch zur Verfügung stehen:

Methode	Pfad	Kurzbeschreibung
GET	`/`	Health-Check – bestätigt, dass die API läuft
GET	`/sessions`	Liste aller Sessions (aktiv & beendet)
GET	`/sessions/{session_id}`	Details einer einzelnen Session
POST	`/sessions`	Neue Session anlegen (Name, Ort, Typ)
PUT	`/sessions/{session_id}/end`	Session beenden und Datenerfassung stoppen
GET	`/sessions/{session_id}/data`	Erhalte Messungen der Session
GET	`/sessions/{session_id}/data/since/{timestamp}`	Messungen einer Session ab Zeitstempel
GET	`/live-data`	Einmalige Momentaufnahme der aktuellen Messwerte
GET	`/devices/status`	Status von Ventilator & Diffusor (TP100)
PUT	`/devices/ventilator/control`	Ventilator ON / OFF / AUTO
PUT	`/devices/diffusor/control`	Diffusor ON / OFF / AUTO
GET	`/air-quality/colors`	Farbcodes für Ratings
GET	`/current-session-info`	Daten zur aktiven Session

API Server starten

```
uvicorn api:app --host 0.0.0.0 --port 8000
```

Dieser Befehl startet das FastAPI-Backend mit dem ASGIServer **Uvicorn** und macht die API im lokalen Netzwerk unter Port 8000 verfügbar.
ASGI (Asynchronous Server Gateway Interface) ist der moderne Standard für asynchrone PythonWebanwendungen und ermöglicht u. a. WebSockets und gleichzeitige Verbindungen.

Session-Management im Code - PP

---

1. Session-Beginn und -Beendigung (API-Workflow)

Das Projekt unterstützt ein vollautomatisiertes Session-Management, sowohl für die Kommandozeile als auch über die REST-API (z. B. für das Web-Frontend).

Start einer Session

- **Per API** (POST `/sessions`):
  - Der User (CLI oder Frontend) gibt einen eindeutigen Session-Namen, optional einen Ort, sowie den Typ („indoor“ oder „outdoor“) an.
  - Die Funktion `start_session()` prüft, ob schon eine offene Session mit diesem Namen existiert:
    - **Falls ja:** Die vorhandene Session wird weiterverwendet, Messwerte werden angehängt.
    - **Falls nein:** Eine neue Session wird mit aktuellem Startzeitpunkt, Name, Ort, Typ angelegt und als aktiv markiert.
  - Die aktuell aktive Session-ID wird global gemerkt (`current_session_id`), sodass alle Messwerte automatisch korrekt zugeordnet werden.
- **Typischer Aufruf (CLI oder API):**

```
# Beispiel (Frontend oder automatisiert)
POST /sessions
{
    "name": "Wohnzimmer Abend",
    "location": "EG",
    "session_type": "indoor"
}
```

- *(Im CLI-Workflow wäre es äquivalent zu:)*

```
python main.py -s "Wohnzimmer Abend" -l "EG" --type indoor
```

Beenden einer Session

- **Per API** (PUT `/sessions/{session_id}/end`):
  - Die Funktion `end_session()` trägt den Endzeitpunkt in die Datenbank ein.
  - Die Session ist damit geschlossen und kann nicht erneut fortgeführt werden.
  - Nach dem Beenden wird `current_session_id` wieder auf `None` gesetzt.

Session-Fortsetzung und Duplikatvermeidung

- Wird versucht, eine Session mit gleichem Namen neu zu starten, prüft die Datenbanklogik, ob bereits eine aktive (nicht beendete) Session existiert:
  - **Falls ja:** Die Messwerte werden weiterhin unter dieser Session-ID gespeichert.
  - **Falls nein:** Es wird eine neue Session mit diesem Namen angelegt.

---

2. Vorteile der Session-Verwaltung

- **Daten werden immer eindeutig zugeordnet** und bleiben gruppierbar (z. B. alle Messwerte im "Wohnzimmer" lassen sich gemeinsam auswerten).
- **Sessionnamen sind eindeutig** (UNIQUE-Constraint) – ein versehentliches doppeltes Anlegen wird unterbunden.
- **Jeder Session kann ein Ort (location) und Typ (indoor/outdoor) zugewiesen werden.**
- **Bequeme Weiterverarbeitung**: Die Session-ID wird für jeden Messwert automatisch mitgespeichert.

---

3. Beispiele aus dem Code

Start einer Session (API):

```
@app.post("/sessions", response_model=SessionResponse)
async def create_session(session: SessionCreate):
    global current_session_id
    conn = get_db_connection()
    session_id = start_session(
        conn,
        session.name,
        session.location,
        session.session_type
    )
    current_session_id = session_id
    ...
```

Session-Ende (API):

```
@app.put("/sessions/{session_id}/end")
async def end_session_endpoint(session_id: int):
    global current_session_id
    conn = get_db_connection()
    end_session(conn, session_id)
    if current_session_id == session_id:
        current_session_id = None
    ...
```

Zusammengefasst:

- Das Session-Management sorgt für klare Datentrennung, verhindert Namensdopplungen und ermöglicht eine lückenlose Messhistorie.
- Sessions können bequem per API oder CLI gestartet und beendet werden – ein automatisiertes Handling aller Zuordnungen ist gewährleistet.

Datenbank-Design & Implementierung - PP

---

1. Was sind „Sessions“? – Konzept und Nutzen

Sessions bezeichnen im Projekt jeweils einen definierten Zeitraum, in dem eine zusammenhängende Messreihe stattfindet.
Eine Session bündelt alle Messwerte, die zu einer bestimmten Umgebung, einem bestimmten Raum, Experiment oder Testablauf gehören.

Hintergrund und Motivation:

- Sensoren werden oft an unterschiedlichen Orten (z. B. Wohnzimmer, Schlafzimmer, Außenbereich) oder für verschiedene Experimente verwendet.
- Ohne Sessions wären alle Messwerte in der Datenbank nicht eindeutig zuordenbar – eine Auswertung nach Ort, Zeitraum oder Test würde unmöglich bzw. sehr aufwendig.

Eine Session umfasst immer:

- **Name:** Klar zuordnungsbare Bezeichnung (z. B. „Wohnzimmer-Abend“, „Outdoor-Test Juli“)
- **Location:** Optional – genauere Beschreibung des Standorts
- **Session-Typ:** Steuert, ob das Indoor- oder Outdoor-Regelwerk zur Bewertung verwendet wird
- **Start- und End-Zeitpunkt:** Automatisch durch das Skript gesetzt, um den Zeitbereich der Messung eindeutig abzubilden
- **Verknüpfte Messwerte:** Alle Luftdaten (Feinstaub, Temperatur, etc.), die während der Session erfasst werden

Ablauf im System:

- Beim Start einer neuen Messung gibt der Nutzer Name, Ort und Typ an (z. B. per CLI-Parameter).
- Das Skript legt (wenn nötig) eine neue Session in der Datenbank an und merkt sich die ID.
- Alle im weiteren Verlauf eingelesenen Sensorwerte werden dieser Session-ID zugeordnet.
- Am Ende der Messung wird die Session per Zeitstempel abgeschlossen.

Vorteile des Session-Konzepts:

- Messdaten bleiben immer klar gruppiert und nachvollziehbar (z. B. für Reports, Diagramme, Export)
- Parallele oder aufeinanderfolgende Versuchsreihen können sauber unterschieden werden
- Session-Management verhindert doppelte Namen und damit Datenchaos in der Datenbank

---

2. Aufbau und Tabellenstruktur

Das Projekt nutzt eine SQLite-Datenbank (`air_quality.db`) im Unterordner `database/`.
Die Datenbank besteht aus zwei logisch verknüpften Tabellen:

sessions

Speichert Metadaten zu einzelnen Mess-Sessions.

- `id` (INTEGER, PRIMARY KEY, AUTOINCREMENT): Eindeutige Session-ID
- `name` (TEXT, UNIQUE, NOT NULL): Bezeichnung der Messreihe (muss eindeutig sein)
- `location` (TEXT): Optionaler Ortsname (z.B. "Wohnzimmer", "Labor")
- `start_time` (TEXT, NOT NULL): Zeitstempel beim Start der Session
- `end_time` (TEXT): Zeitstempel beim Beenden der Session
- `session_type` (TEXT, NOT NULL): "indoor" oder "outdoor" für das Regelwerk

air_data

Speichert alle ausgelesenen Messdaten und referenziert immer eine Session.

- `id` (INTEGER, PRIMARY KEY, AUTOINCREMENT): Eindeutige Messwert-ID
- `timestamp` (TEXT, NOT NULL): Zeitpunkt der Messung
- `pm1_0`, `pm2_5`, `pm10` (REAL): Feinstaubwerte aus der Messzeile
- `raw_line` (TEXT): Die vollständige, rohe Sensorzeile (zur späteren Analyse)
- `session_id` (INTEGER, NOT NULL): Fremdschlüssel auf `sessions.id`

```
# CREATE TABLE Beispiel aus db_helper.py
c.execute('''
    CREATE TABLE IF NOT EXISTS sessions (
        id            INTEGER PRIMARY KEY AUTOINCREMENT,
        name          TEXT    NOT NULL UNIQUE,
        location      TEXT,
        start_time    TEXT    NOT NULL,
        end_time      TEXT,
        session_type  TEXT    NOT NULL
    );
''')
c.execute('''
    CREATE TABLE IF NOT EXISTS air_data (
        id          INTEGER PRIMARY KEY AUTOINCREMENT,
        timestamp   TEXT    NOT NULL,
        pm1_0       REAL,
        pm2_5       REAL,
        pm10        REAL,
        raw_line    TEXT,
        session_id  INTEGER NOT NULL,
        FOREIGN KEY(session_id) REFERENCES sessions(id)
    );
''')
```

---

3. Beziehungen und Regeln

Eindeutige Sessionnamen:

- Jede Session hat einen eindeutigen Namen (UNIQUE).
  Ein erneuter Start mit gleichem Namen führt dazu, dass Messwerte der selben Session-ID zugeordnet werden.

- **Session-Typ:**
  `"indoor"` oder `"outdoor"` steuert das Bewertungsregelwerk.
- **Fremdschlüssel:**
  Messwerte sind immer einer Session zugeordnet (`session_id` als FOREIGN KEY).

---

4. Datenhaltung und Insert-Logik

Session-Handling:

- Neue Session wird über `start_session` angelegt:
  - Prüft, ob der Name schon existiert; falls ja, wird die Session-ID zurückgegeben.
  - Legt sonst eine neue Session an.
- Beenden einer Session (`end_session`) trägt einen Endzeitpunkt ein.

Messwert-Speicherung:

- Über `insert_data` werden Messwerte mit Zeitstempel, extrahierten PM-Werten und der kompletten Rohzeile in `air_data` gespeichert.
- Jeder Datensatz bekommt den zugehörigen `session_id`.

Beispiel (aus `save_data.py`):

```
def insert_data(conn: sqlite3.Connection, raw_line: str, session_id: int):
    ...
    c.execute('''
        INSERT INTO air_data
          (timestamp, pm1_0, pm2_5, pm10, raw_line, session_id)
        VALUES (?, ?, ?, ?, ?, ?)
    ''', (ts, pm1, pm2p5, pm10, raw_line, session_id))
    ...
```

---

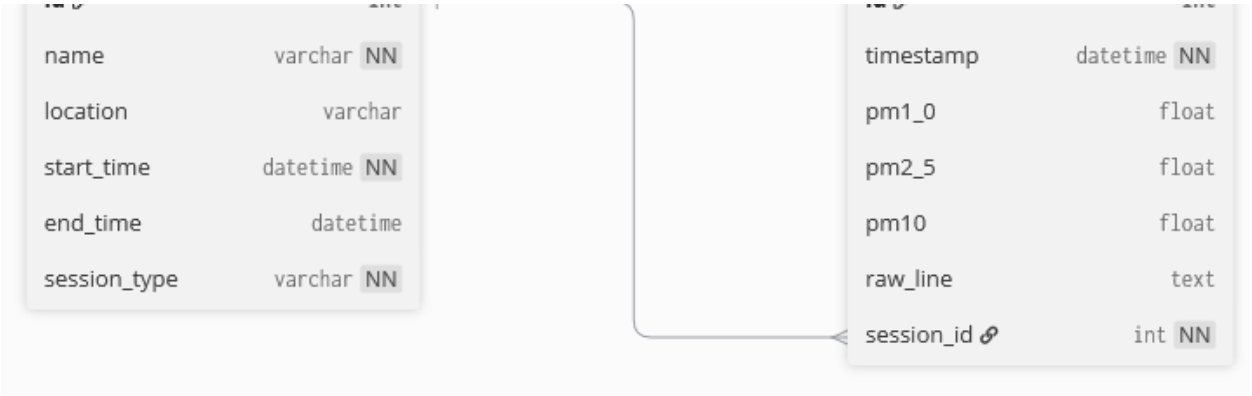
5. Fehlerbehandlung und Migration

- **Fehlerbehandlung:**
  Jeder Datenbankzugriff ist mit `try/except` abgesichert. Fehler werden auf der Konsole protokolliert.
  Schema-Änderungen/Migration:
- Bei Änderungen an den Tabellen (z. B. neue Spalten wie `session_type`) wird empfohlen, die alte Datenbankdatei zu löschen – beim nächsten Start legen die Python-Skripte die DB automatisch neu mit aktuellem Schema an.

---

6. Visualisierung





Zusammengefasst:

- Die SQLite-Datenbank ist übersichtlich, modular und schützt vor Redundanzen (z. B. durch UNIQUE-Constraint auf `name`).
- Messdaten und Sessions sind sauber voneinander getrennt und durch einen Fremdschlüssel verknüpft.
- Die Einbindung neuer Felder (wie weitere Sensorkanäle) ist leicht erweiterbar.

Device Steuerung durch WLAN-Steckdosen - SK

Ziel

Das Ziel ist es, Geräte basierend auf der aktuellen Luftqualität automatisch zu steuern. Dazu haben wir einen Ventilator und einen Aroma Diffusor bzw Luftbefeuchtiger, die je über eine WLAN-Steckdose gesteuert werden können.

Umsetzung

Python Library

Die WLAN-Steckdosen sind das Modell TP-Link Tapo P100. Für diese gibt es einige Python Libraries, die man in Betracht ziehen kann, wie z.B. PyP100, pytapo. Allerdings funktionierte die Standardinstallation per pip zum Zeitpunkt der Implementierung nicht, da unsere Steckdosen bereits die Firmware-Version 1.2.5 mit neuer Authentifizierungsmethode verwendeten – welche von den offiziellen Libraries noch nicht unterstützt wurde. Dieses Fork von `PyP100` hat schließlich zum Erfolg geführt: https://github.com/almottier/TapoP100

tp100_controller.py (WLAN-Steckdosen Controller)

Dieses Modul stellt eine Python Klasse `TP100Controller` zur Verfügung, mit der eine P100 WLAN-Steckdose gesteuert werden kann.

Dazu kann man sich mit dem Gerät mithilfe der Tapo App Anmeldedaten verbinden, das Gerät ein- und ausschalten und den Status abfragen, ob das Gerät eingeschaltet ist.

```
from PyP100 import PyP100

# Daten aus Tapo App
TP100_IP = ""
TAPO_EMAIL = ""
TAPO_PASSWORD = ""

class TP100Controller:
    def __init__(self, ip, email, password):
        self.p100 = PyP100.P100(ip, email, password)
        self._login()

    def _login(self):
        try:
            self.p100.handshake()
            self.p100.login()
        except Exception as e:
            raise RuntimeError(f"Verbindung fehlgeschlagen: {e}")

    def turn_on(self):
        self.p100.turnOn()

    def turn_off(self):
        self.p100.turnOff()

    def get_status(self):
        info = self.p100.getDeviceInfo()
        return "on" if info.get('device_on', False) else "off"

    def get_name(self):
        return self.p100.getDeviceName()
```

air_control_rules.py (Regeln zur Steuerung)

In diesem Modul sind Regeln definiert, wann der Ventilator und der Diffusor an und aus sein sollte. Dazu wurden wieder die Messwerte (*measurements*), die auch für das Air Quality Rating verwendet werden, jedoch ohne Temperatur, da dieser Faktor nicht durch unsere Geräte steuerbar ist, benutzt.

Es gibt für jedes Gerät zwei Methoden.

Ventilator

Die Methode needs_ventilation beschreibt, wann der Ventilator eingeschaltet sein sollte. Das ist der Fall, wenn mindestens einer der vier Werte (PM2.5, CO2eq, VOCAQI, HUMID) mit *schlecht* oder *sehr schlecht* bewertet ist. Wenn der SessionType *OUTDOOR* ist, aktiviert sich der Ventilator nicht, weil eine aktive Lüftung im Außenbereich keinen Einfluss auf die Umgebungsluft hat. Auch erwähnt sollte sein, dass ein Ventilator an sich die Luftqualität nicht verbessert, da er im Grunde nur die Luft umwirbelt. Der Gedanke hier ist, dass man den Ventilator mit lüften verbindet, um frische Luft schneller in den Raum zu bringen.

ventilation_not_needed schaltet den Ventilator aus, wenn alle vier Werte *sehr gut* oder *gut* sind. Eine *mäßige* Bewertung wurde bewusst in beiden Methoden nicht beachtet, da dies keinen zwingenden Handlungsbedarf erfordert und um ein häufiges Umschalten des Ventilators zu vermeiden.

Diffusor

Aus demselben Grund wie beim Ventilator wird der Diffusor bei Outdoor-Sessions nicht geschaltet. Bei normaler Nutzung eines Aroma Diffusors (wenn man mehr als nur Wasser benutzt) sollte man vorsichtig sein, da dieser die Luftqualität sogar verschlechtern kann.

Darauf wurde in der Steuerung geachtet und deswegen ist die Vorraussetzung für die Aktivierung des Aroma Diffusor, dass die Luftqualität an sich gut ist, d.h. wenn PM2.5 schlechtestens *mäßig* ist und VOCAQI schlechtestens *gut* ist, damit der Aroma Diffusor keine Belastung ist. Wenn diese Vorraussetzung erfüllt ist und die Luft zu trocken ist (< 30% Luftfeuchtigkeit), dann gibt diffusor_should_be_on *True* zurück.

Wenn also die Luftfeuchtigkeit *sehr gut* ist oder VOCAQI oder PM2.5 *schlecht* bis *sehr schlecht* ist, wird der Aroma Diffusor deaktiviert.

Testing & Qualitätssicherung - PP

1. Teststruktur und -aufbau

Das Projekt verwendet das **pytest-Framework** für Unit-Tests. Alle Tests befinden sich gesammelt im Ordner `tests/`:

- **test_db_helper.py**:
  Prüft die Datenbankinitialisierung, Session-Start/-Ende, Duplikatvermeidung usw.
- **test_save_data.py**:
  Testet das Schreiben von Messwerten und Logfiles sowie das Insert in die Datenbank.
- **test_read_sensor_data.py**:
  Simuliert das Lesen von Sensordaten, DummySerial, Parsing und Extraktion der Werte.

Alle Tests nutzen temporäre Datenbanken und Dateien, sodass beim Testen keine echten Produktionsdaten überschrieben werden.

---

2. Testausführung

Um alle Tests auszuführen:

1. **Virtuelle Umgebung aktivieren (falls genutzt):**
   `source venv/bin/activate`

2. **In das Projekt-Hauptverzeichnis (eine Ebene über `tests/`) wechseln:**
   `cd <projekt-root>`

3. **Pytest starten:**
   `pytest -q`

Pytest sucht dabei automatisch nach allen Dateien, die mit `test_` beginnen, und führt alle enthaltenen Tests aus.

---

3. Besondere Testmethoden und Tools

- Es werden **Fixtures** für temporäre Datenbanken (`tmp_path`, `monkeypatch`) verwendet, um ein sauberes Testumfeld zu gewährleisten.
- Für die Sensorik wird eine DummySerial-Klasse mit monkeypatch als Ersatz für den echten Sensor genutzt, damit Tests unabhängig von echter Hardware laufen können.
- Es wird geprüft, dass Logfiles geschrieben werden und dass die wichtigsten Ausgaben tatsächlich auf der Konsole erscheinen.

---

4. Beispiel-Testfall (test_save_data.py)

```
def test_insert_data_and_log(tmp_path, conn, capsys, monkeypatch):
    raw = "PM1.0,1.1,PM2.5,2.2,PM10,3.3,TEMP,20,HUMID,50"
    sid = start_session(conn, "SessionX", None)
    insert_data(conn, raw, sid)
    cur = conn.cursor()
    cur.execute("SELECT pm1_0, pm2_5, pm10, session_id FROM air_data")
    pm1, pm2p5, pm10, sess = cur.fetchone()
    assert pm1 == 1.1
    assert pm2p5 == 2.2
    assert pm10 == 3.3
    assert sess == sid
```

---

5. Typische Fehlerbilder beim Testen und Lösungen

- **ImportError für lokale Module:**
  → Tests immer aus dem Projekt-Root starten, `__init__.py`-Dateien in jedem Ordner anlegen.
- **DB wird nicht überschrieben:**
  → Über `tmp_path` und Monkeypatch werden temporäre Datenbanken verwendet.
- **Hardwareabhängige Fehler:**
  → DummySerial und Fixtures ersetzen echte Sensorik beim Testen.

---

6. Aktualisieren der requirements.txt

Nach Installation neuer Pakete:
`pip freeze > requirements.txt`

→ So sind alle Test-Abhängigkeiten dokumentiert und reproduzierbar

---

Mit diesem Testkonzept ist die Codequalität abgesichert, und Fehler werden frühzeitig im Entwicklungsprozess erkannt.

Air Quality Rating - SK

Ziel

Das Ziel war es die Luftqualität auf einer Skala bewerten zu können anhand der gelesenen Daten des Sensors.

Umsetzung

Aufteilung in 5 Kategorien

- SEHR_SCHLECHT
- SCHLECHT
- MÄSSIG
- GUT
- SEHR_GUT

Konzentrieren auf 5 Messwerte. Die Auswahl basiert auf gesundheitsrelevanter Aussagekraft, Sensorverfügbarkeit und Praxisnähe

- **PM2.5** – Wichtigster Indikator für Feinstaubbelastung laut WHO. Tiefes Eindringen in die Lunge, stark gesundheitsschädlich.
- **CO2eq** – Zeigt Luftwechsel und Frischluftzufuhr an. Hohe Werte beeinträchtigen Konzentration und Wohlbefinden.
- **VOCAQI** – Umfasst schädliche flüchtige organische Verbindungen (VOCs) in einem verständlichen Index.
- **Temperatur** – Beeinflusst thermischen Komfort. Wegen möglicher Sensorverzerrung nur gering gewichtet.
- **Luftfeuchtigkeit** – Zu hohe oder zu niedrige Werte fördern Schimmel oder Reizungen. Wichtig für Komfort und Gesundheit

Trennung von Indoor und Outdoor

Da bei der Erstellung der Sessions *indoor* oder *outdoor* angegeben werden kann, ist es sinnvoll die Bewertung für Indoor und Outdoor getrennt zu behandeln. Daher gibt es andere Schwellen der einzelnen Parameter. Dazu gleich mehr.

Auswählen der Schwellenwerte zum Einteilen in die obigen 5 Kategorien

Die Schwellenwerte orientieren sich an international anerkannten Normen, Richtlinien und wissenschaftlichen Empfehlungen.

Dazu wurde primär die Indoor Empfehlungen betrachtet basierend u.a. auf:

- WHO Air Quality Guidelines (2021) für PM2.5
- ASHRAE Standard 55 für Temperatur und Luftfeuchtigkeit
- AirNow Air Quality Index (AQI) für VOCAQI
- Umweltbundesamt & DIN EN 13779 für CO2-Konzentration

Die Outdoor-Schwellen wurden angepasst, um typische Außenbedingungen realistisch abzubilden:

- Lockerere Grenzen bei Temperatur & Luftfeuchtigkeit (größere Schwankungen)
- Strengere CO2eq-Werte, da Frischluft als Referenz dient
- Höhere VOCAQI-Toleranz, da VOCs außen weniger kontrollierbar sind
- PM2.5 ebenfalls angepasst an WHO-Richtlinie für 24h-Grenzwerte

So ergaben sich diese Schwellenwerte:

Indoor

Parameter	Sehr gut	Gut	Mäßig	Schlecht	Sehr schlecht
PM2.5 (µg/m³)	0–5	>5–10	>10–15	>15–25	>25
COeq (ppm)	≤600	601–800	801–1000	1001–1400	>1400
VOCAQI	≤50	51–100	101–150	151–200	>200
Temperatur (°C)	20–26	17–29	14–32	11–35	<11 oder >35
Luftfeuchtigkeit (%)	40–60	30–65	20–75	10–85	<10 oder >85

Outdoor

Parameter	Sehr gut	Gut	Mäßig	Schlecht	Sehr schlecht
PM2.5 (µg/m³)	0–10	>10–15	>15–25	>25–35	>35
COeq (ppm)	≤450	451–800	801–1200	1201–2000	>2000
VOCAQI	≤100	101–200	201–300	301–400	>400
Temperatur (°C)	18–26	12–30	5–33	0–36	<0 oder >36
Luftfeuchtigkeit (%)	40–70	30–80	20–90	10–95	<10 oder >95

Gesamtwertung

Zunächst wird von jedem der 5 Messwerte die eigene Bewertung gebildet anhand der Schwellenwerte. Anhand dieser Bewertungen dann die Gesamtwertung der Luftqualität.

Alter Ansatz

Anfangs wurde die Gesamtwertung so implementiert, dass sie von allen 5 Messwerten den schlechtesten genommen hat, nach der Idee, dass wenn ein Wert extrem schlecht z.B. PM2.5 sehr hoch ist, es insgesamt gefährlich ist und so auch die Luftqualität extrem schlecht ist.

Allerdings wurde der Sensor von sich aus immer sehr warm, weshalb die Temperatur immer *Sehr schlecht* war, und demnach auch die Luftqualität *Sehr schlecht* war, obwohl es in Realität gute Luftqualität war.

Verbesserter Ansatz

Um dieses Problem zu lösen, wurden Gewichtungen der Messwerte eingeführt.

Parameter	Gewicht
PM2.5	40 %
COeq	25 %

VOCAQI	20 %
Luftfeuchtigkeit	10 %
Temperatur	5 %

Jeder Parameter trägt entsprechend seiner Relevanz zur Gesamtbewertung bei. So haben kritische Werte wie PM2.5 mehr Einfluss als z. B. Temperatur.

Die Einzelbewertungen werden dabei mit ihrem Gewicht multipliziert, aufsummiert und anschließend gerundet – so entsteht ein ausgewogener Mittelwert als Gesamturteil.

Luftsensor auslesen - SK

Als Luftsensor verwenden wir den fortgeschrittenen *Canāree I5* Luftsensor von Piera Systems.
Er kombiniert präzise Partikelzählung mit zusätzlichen Umwelt- und Gasparametern in einem kompakten Gerät.
Der Canāree I5 kann Partikel bereits ab 0,1 µm messen – ein Bereich, den die meisten herkömmlichen Feinstaubsensoren nicht abdecken.

Der Sensor liefert jede Sekunde eine solche Key,Value Rohzeile über USB. Wir verwenden `pyserial` um die Daten mit Python einzulesen.

PC0.1,154397,PC0.3,61758,PC0.5,23998,PC1.0,1048,PC2.5,0,PC5.0,2,PC10,0,PM0.1,0.14128961,PM0.3,1.66720926,PM0.5,4.41222128,PM1.0,5.37196085,PM2.5,5.37196085,PM5.0,5.43639578,PM10,5.45372612,TEMP,23.91,HUMID,49.21,ATM,981.59,VOCAQI,25.00,bVOCeq,0.50,GASest,0.00,CO2eq,500.00,CNR-I5-24I000241

Sensorwerte und Bedeutung

Die vom Canāree I5 gelieferten Rohdaten bestehen aus Schlüssel-Wert-Paaren im Format `KEY,VALUE`. Nachfolgend eine Übersicht der Werte:

Schlüssel	Bedeutung
PC0.1 – PC10	Partikelanzahl ≥ x µm pro 0,1 L Luft
PM0.1 – PM10	Partikelmassekonzentration in µg/m³
TEMP	Temperatur in °C
HUMID	Luftfeuchtigkeit in %
ATM	Luftdruck in hPa
VOCAQI	Luftqualitätsindex basierend auf VOC (0–500)
bVOCeq	geschätzte VOC-Konzentration in ppm
GASest	allgemeine Gasbelastung (modellabhängig)
CO2eq	CO-Äquivalent in ppm

Der letzte Eintrag in der Datenzeile ist die Seriennummer des Sensors.
Die Werte werden als fortlaufender CSV-String über die serielle Schnittstelle übertragen.

read_sensor_data.py

In unserem Projekt sorgt `read_sensor_data.py` für das Einlesen des Sensors.

SensorReader – Wichtige Codeausschnitte

Nachfolgend sind die zentralen Methoden von `read_sensor_data` aufgeführt. Die in der Implementierung vorhandenen Debug-Ausgaben wurden zur Übersichtlichkeit weggelassen.

Initialisierung und Verbindung

Initialisiert den Sensor. `/dev/ttyUSB0` ist dabei der Port, an dem der Sensor am Raspberry Pi angeschlossen ist. Diesen kann man über die Parameter bei der Objekterstellung auch anpassen. `close` schließt die Verbindung wieder.

```
class SensorReader:
    def __init__(self, port='COM4', baudrate=115200):
       try:
            self.ser = serial.Serial(port, baudrate, timeout=1)
        except serial.SerialException as e:
            self.ser = None
    …
    def close(self):
        self.ser.close()
```

Einlesen roher Zeile

Liest eine Datenzeile vom Sensor und liefert `None`, wenn keine Daten anliegen.

```
    def lese_daten(self):
        line = self.ser.readline().decode('utf-8', errors='ignore').strip()
        return line if line else None
```

Parsen in Dict

Die Zeile wird an Kommas aufgeteilt und in ein PythonDict umgewandelt, um über die Keys direkten Zugriff auf die Messwerte zu ermöglichen. Eine ungerade Anzahl an Teilen signalisiert eine GeräteID am Ende, die mit `parts.pop()` entnommen wird. Zusätzlich wird ein ISOTimestamp eingefügt.

parse_sensor_line

```
def parse_sensor_line(self, line):
    parts = [p.strip() for p in line.split(',')]

    if len(parts) < 2:
        return None

    result = {}
    result['timestamp'] = datetime.now().isoformat()

    if len(parts) % 2 != 0:
        device_id = parts[-1]
        result['device_id'] = device_id
        parts = parts[:-1]

    i = 0
    while i < len(parts) - 1:
        key = parts[i]
        value = parts[i + 1]

        try:
            value = float(value)
            if value.is_integer():
                value = int(value)
        except ValueError:
            pass

        result[key] = value
        i += 2

    return result
```

Weitere bereitgestellte Methoden

- `get_rating_measurements(raw_line)`: Extrahiert die 5 Key-Measurements PM2.5, CO2eq, VOCAQI, TEMP, HUMID, die für die Bewertung der Luftqualität und Steuerung der Geräte notwendig sind.
- `speichere_json(daten, file)`: Hängt jedes Dict als eigene JSONZeile an `file` an für Logging.
- `lese_daten_parsed:` Liest Daten vom Sensor und gibt sie direkt als Python-Dict zurück.

Kurzbeispiel der Verwendung

Instanziiert das SensorReaderObjekt, öffnet die serielle Schnittstelle und liefert alle Messwerte als strukturiertes PythonDict (inkl. Timestamp) und schließt die Verbindung.

```
from sensorik.read_sensor_data import SensorReader

reader = SensorReader()
raw = reader.lese_daten()
if raw:
    data = reader.parse_sensor_line(raw)
    print(data)
reader.close()
```

Frontend - MK

1. Motivation

Ein reaktives Web-UI, das Messwerte übersichtlich darstellt, Sessions bedienbar macht und Ventilator/Diffusor per Klick steuert und die Daten exportierbar macht(z.B. als pdf) – optional auch ohne laufendes Backend (Mock-Modus).

2. Gesamtübersicht (Frontend-intern)

Technologie-Stack

- **Framework:** React (mit Hooks), Vite als Build-Tool
- **Styling:** Tailwind CSS
- **HTTP/WebSocket:** Axios (REST), WebSocket
- **Tests:** Jest + React Testing Library

Frontend-Architektur

```
src/
 components/        -> UI-Bausteine (z.B. DeviceControlPanel, SessionManager)
 services/          -> API-Zugriff (fetchDeviceStatus, fetchSessions, ...)
 hooks/             -> Wiederverwendbare Logik (z.B. useLiveStream, useSessionDelta)
 utils/             -> Hilfsfunktionen (Export, Env-Shim)
 ...
```

- **services** kapseln alle Backend-Aufrufe → leicht zu mocken.
- **components** sind primär zustandsarm, beziehen Daten über Hooks/Services.
- **tests** spiegeln die Ordnerstruktur wider und testen Komponenten & Services isoliert.

Zusammenspiel

1. Komponente mountet → ruft Service-Funktion(en) aus `src/services/api`.
2. Daten kommen (Mock/Backend) → State aktualisiert → UI rendert.
3. User-Aktionen (Buttons) lösen Service-Calls aus → State-Update.

3. Wichtige Abläufe

3.1 Gerätesteuerung (Ventilator/Diffusor)

Sequenz :

1. `DeviceControlPanel` mounted → `setTimeout(loadStatus, 1000)`

2. `loadStatus()` ruft `fetchDeviceStatus()`
3. UI zeigt Status-Badges & Buttons. Klick auf „An/Aus/Auto“ → `controlVentilator|controlDiffusor()`
4. Busy-Flag (`loadingKey`) deaktiviert Button kurz; nach Response wird Status aktualisiert.

3.2 Session-Verwaltung

1. `SessionManager` lädt Sessions via `fetchSessions()`
2. Start/Ende via `startSession()` / `endSession()`
3. Datenexport in `exportSessionData()` (CSV/JSON) über Blob/Download-Link.

3.3 Live-Daten (falls genutzt)

1. Hook `useLiveStream` öffnet WebSocket oder Polling zu `/live-data`
2. State wird fortlaufend aktualisiert → Charts/Badges updaten sich.

---

4. Wichtige Codestellen

4.1 `src/components/DeviceControlPanel.jsx`

- Verzögertes Laden via `setTimeout` + `useEffect`
- Buttons mit Busy-State & Auto/On/Off-Logik

```
useEffect(() => {
  const timer = setTimeout(loadStatus, 1000);
  return () => clearTimeout(timer);
}, [loadStatus]);
```

4.2 `src/services/api.js`

- Zentraler Zugriffspunkt für REST-Calls (Status holen, Sessions, etc.)
- Erlaubt Modul-Mocking in Tests (`jest.mock('src/services/api', ...)`).

4.3 `src/utils/exportSession.js`

- Clientseitiger Export (CSV/JSON) ohne Server-Roundtrip.
- Nutzung von Blob + `URL.createObjectURL` + temporärem <a>-Element.

4.4 `src/utils/envShim.js`

- Fallback für ENV-Zugriff in Tests (wenn `import.meta.env` nicht verfügbar/mokbar war). Kann entfernt werden, wenn Tests direkt `import.meta.env` mocken.

---

5. Installation / Einrichtung

```
npm install        # Dependencies
npm run dev        # Dev-Server starten
```

Öffne die ausgegebene URL (z. B. http://localhost:5173).

ENV-Konfiguration

```
VITE_USE_MOCK=true              # Mock-Daten einschalten
VITE_API_URL=http://localhost:8000   # Backend-URL im Echtbetrieb
VITE_WS_URL=ws://172.20.10.4:8000
Bzw. Starten des Docker compose (siehe Deployment Dokumentation)
Über Docker Container auch Zugriff von anderen Geräten auf das Frontend über (172.20.10.4:3000) a
```

---

6. Tests & Qualitätssicherung

```
npm run test          # Jest im Watch-Modus
npm run test:once     # einmalig (CI)
```

- Komponenten-Tests mit React Testing Library
- Fake-Timer bei Komponenten, die `setTimeout` nutzen (z. B. DeviceControlPanel)
- API-Funktionen werden über `jest.mock('src/services/api')` isoliert.

---

7. Architekturelle Entscheidungen (Frontend)

- **React + Vite**: schnelle Dev-Experience, Hot Module Reloading
- **Tailwind CSS**: schnelle UI-Prototypen, konsistente Utility-Klassen
- **Mock-Modus per ENV** statt Hardcoding → flexibles Testen/Demo

---

8. Betriebshinweise

- Frontend ist ein statisches Bundle (`npm run build` → `dist/`) und kann auf jedem Static Host laufen (Netlify, Vercel, Nginx).
- ENV-Variablen müssen zur Build-Zeit gesetzt sein (Vite liest `.env*`).
- Keine regelmäßigen Wartungsaufgaben nötig (keine Serverprozesse).

---

9. Detailierte Frontend-Implementierung

9.1 Services (`src/services/api.js`)

Rolle: zentrale Abstraktionsschicht für REST- und WebSocket-Kommunikation + Mocking.

- **ENV-Schalter:** `USE_MOCK` entscheidet, ob echte Endpunkte oder Mock-Funktionen genutzt werden.
- **Axios-Client mit Retry & Interceptor:**

```
export const http = axios.create({ baseURL: BASE_URL, timeout: 8000 });
axiosRetry(http, { retries: 2, retryDelay: axiosRetry.exponentialDelay, retryCondition: err => ax
http.interceptors.response.use(res => res, err => { console.warn('[HTTP INTERCEPTOR]', err.config
```

- **Mock-Helfer & Normalizer:** Zufallswerte (`rand`), Key-Normalisierung (PM2_5 → PM2.5) und `normalizeLivePayload` sorgen für einheitliche Datenstrukturen.
- **Sessions/History/Devices:** CRUD-ähnliche Funktionen (`fetchSessions`, `startSession`, `fetchSessionData`, `controlVentilator`, ...) sind klar getrennt.
- **WebSocket-Handling (`connectLive`)** mit automatischem Reconnect (bis zu 6 Versuche) und Mock-Interval im Demo-Modus.

9.2 Hooks

`useLiveStream(active)`

- Lädt initial per HTTP (`fetchLiveData`) und hält danach via `connectLive` den State aktuell.
- Rückgabe: `{ live, wsState }` – Komponenten müssen nur diese Werte konsumieren.

```
useEffect(() => {
  if (!active) return;
  fetchLiveData().then(first => mounted && setLive(first));
  cleanupRef.current = connectLive(payload => mounted && setLive(payload), setWsState);
  return () => cleanupRef.current?.();
}, [active]);
```

`useSessionDelta(sessionId, enabled, pollingMs)`

- Lädt initial komplette History (`fetchSessionData`) und pollt danach deltas via `fetchSessionDataSince`.
- Nutzt `setInterval` → im Test werden Fake-Timer eingesetzt.

9.3 Wichtige Komponenten

`Dashboard.jsx`

- **Orchestrator**: bindet Hooks, Services und UI-Kacheln zusammen.
- Verwaltet Sessions, History, Snapshots (live vs. frozen), Export (CSV/JSON/PDF).
- Enthält Helper wie `buildHistoryEntryFromLive` und `triggerDownload`.

`DeviceControlPanel.jsx`

- Lädt Gerätestatus verzögert (1s Timeout) → verhindert unnötige Requests beim Render.
- Buttons setzen Busy-State und aktualisieren Status nach erfolgreichem Request.
- Test erfordert Fake-Timer (`jest.useFakeTimers()` + `advanceTimersByTime`).

```
useEffect(() => {
  const timer = setTimeout(loadStatus, 1000);
  return () => clearTimeout(timer);
}, [loadStatus]);
```

`HistoryChart.jsx`

- Recharts-LineChart mit dynamisch auswählbaren Metriken.
- State `selected` steuert sichtbare Linien, Utility-Funktionen für „Core/Alle/Keine“.

`AirQualitySummary.jsx`

- Zeigt Gesamtluftqualität + Mini-Sparkline (SVG) der letzten 30 Werte.
- Farb-Codierung per Tailwind-Klassenmap (`COLOR_MAP`).

`ParticleChartCard.jsx`

- BarChart (Recharts) für Partikelmasse/-anzahl, „Freeze“-Badge bei historischen Sessions.

`SensorMetricCard.jsx`

- Kleinste UI-Einheit für Einzelwerte, nutzt Farbcodes (`RATING_COLORS`) und Freeze-Indikator.

9.4 Utilities

- `exportSessionData(rows, format)` → CSV/JSON Export via Blob/URL.
- `envShim.js` → Fallback für ENV im Testumfeld; kann durch direktes Mocken von `import.meta.env` ersetzt werden.

9.5 Tests

- **Komponenten-Tests** (RTL): z. B. `DeviceControlPanel.test.jsx` mit Fake-Timern.
- **Hook-Tests** (`renderHook`): `useLiveStream`, `useSessionDelta` → prüfen initiale Ladung & Updates.
- **Service-Tests**: API-Methoden werden komplett gemockt → Struktur & Rückgabewerte werden verifiziert.

```
// Fake-Timer Beispiel
beforeAll(() => jest.useFakeTimers());
afterAll(() => jest.useRealTimers());
await act(async () => { jest.advanceTimersByTime(1000); await Promise.resolve(); });
```

9.6 ENV & Mocking-Strategie

- `.env` steuert Modus & URLs: `VITE_USE_MOCK`, `VITE_API_URL`, `VITE_WS_URL`.
-

- Im Mock-Modus liefern Services synthetische, aber realistisch variierende Daten.
- Vorteil: Frontend ist vollständig demonstrierbar ohne Backend/Hardware.

9.7 Styling & UI-Konzept

- **Tailwind CSS** + eigene Utility-Klassen (z. B. `.aq-card`, `.badge`).
- Heller Hintergrund mit CSS-Gradients, konsistente Komponentenkarten.
- Kleine, konsistente Typografieklassen (`text-xxs`).

9.8 Router & Entry-Point

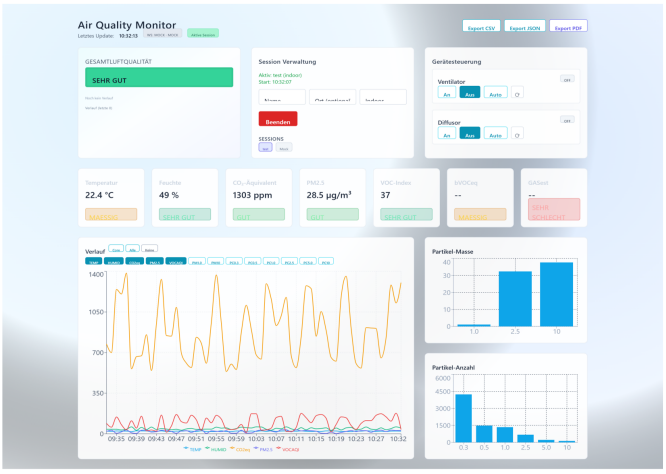
- `App.jsx` nutzt React Router (`/dashboard`).
- `main.jsx` setzt `window.__VITE_ENV__` (Debugging) und rendert `<App />` in `#root`.

---

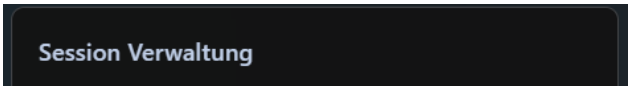
10. ToDos / Weiterer Ausbau (Frontend)

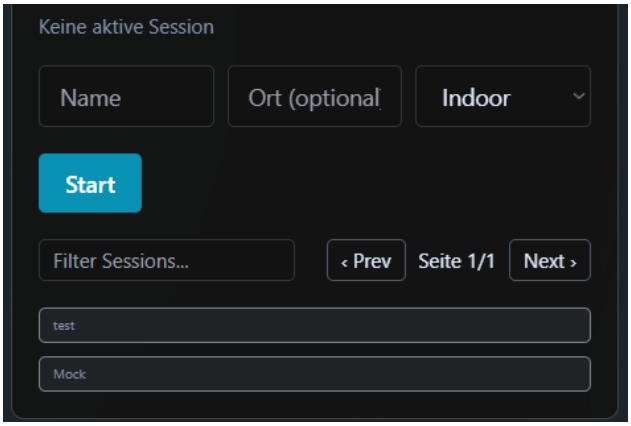
- Umschalten auf Vitest (vereinfachter Test-Runner für Vite) → Babel entfernen.
- WebSocket-Backoff/Retry parametrisierbar machen.
- State-Management zentralisieren (z. B. Zustand/Redux) bei wachsender Komplexität.
- Geräte Hinzufügen in Gerätestuerung
- Hinzufügen weiterer Pages (z.B. Vergleichsseite von Sessions)
- Verbesserung der Barrierefreiheit (Accessibility) (z.B. ARIA-Attribute)
- Pagination und Daten-Navigation verbessern (z.B. Intersection Observer)
- Internationalisierung (z.B. Grad Fahrenheit / Celsius Switch)
- Löschen von Sessions

11. Screenshots/Visualisierungen des Frontends/Komponenten (Mock-Betrieb)

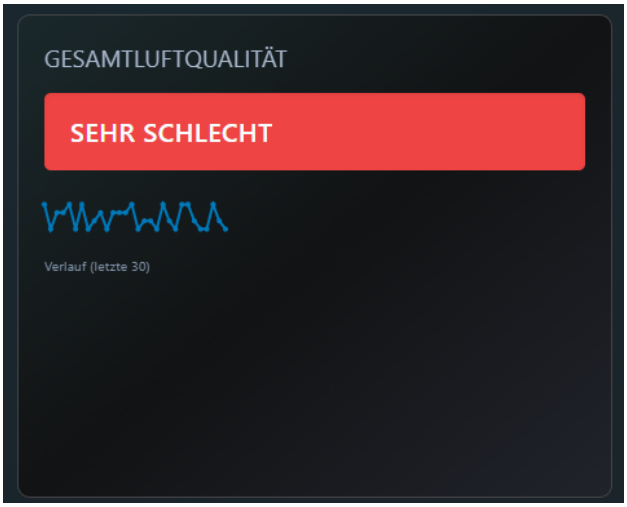


Frontend export funktion (pdf)

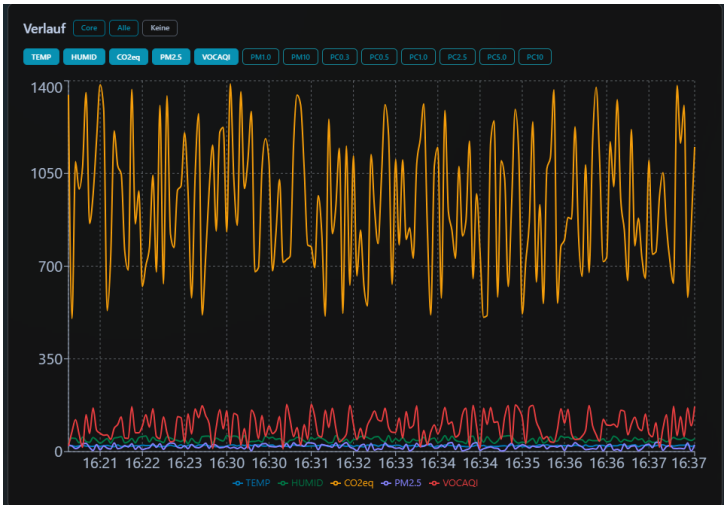




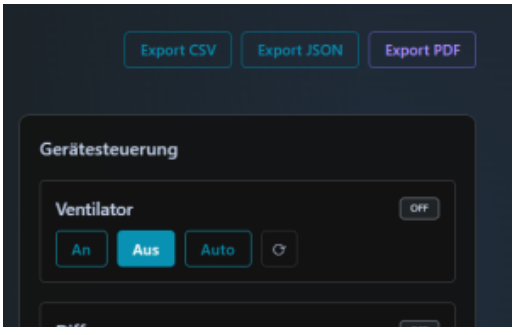
Komponente (SessionManager)

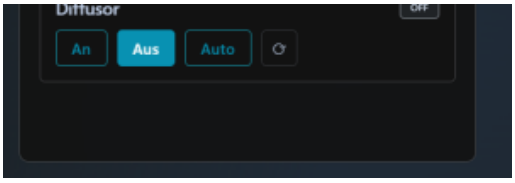


Komponente (AirQualitySummary)

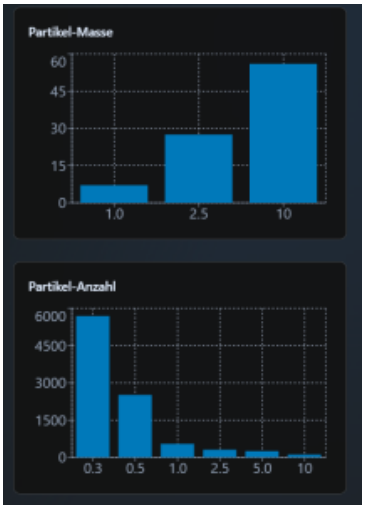


Komponenten (HistoryChart Chart/Card/Filter)





Komponente (DeviceControlPanel)



Komponente (ParticleChartCard)

LED-Ampel & MQTT - CM

Alles erweitern Alles ausblenden

Vorbereitung und Hardware - CM

Da der NeoPixel-Ring als Bauteil ohne direkte Anschlusskabel geliefert wird, mussten an dessen Ports zuerst Kabel angelötet werden und mit einem passenden Pin-Stecker für JumperKabel versehen werden:

Your browser does not support the HTML5 video element

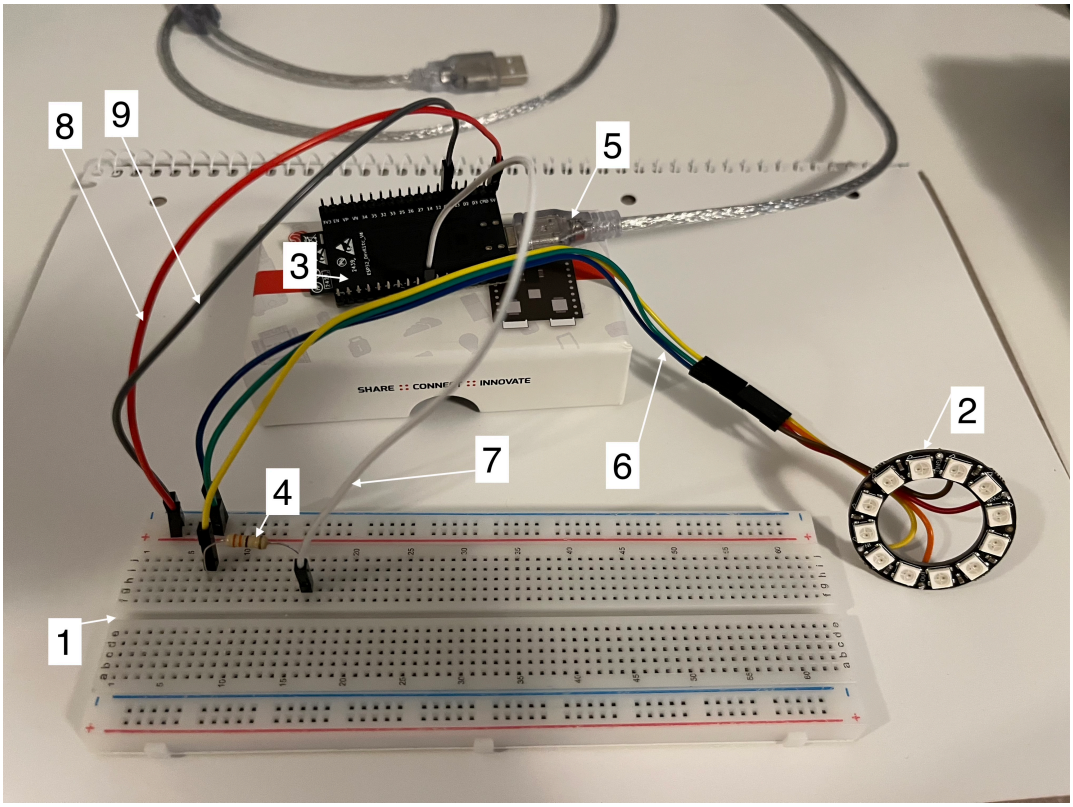
> Vorbereiten der modifizierten Jumper-Kabel

Your browser does not support the HTML5 video element

> Anlöten der vorbereiteten Kabel an die vier Ports des NeoPixel-Rings:
>
> - 2 Kabel zur Stromversorgung (**+ und -**)
> - Data-In-Kabel zur Datenübertragung vom ESP32 an den Pixel-Ring
> - Data-Out-Kabel → wird in unserem Projekt nicht benötigt, aber könnte für zukünftige Erweiterungen von nutzen sein, da hierdurch auch ein Chaining mehrerer NeoPixel-Produkte mit nur einem einzigen Data-In-Port möglich ist.

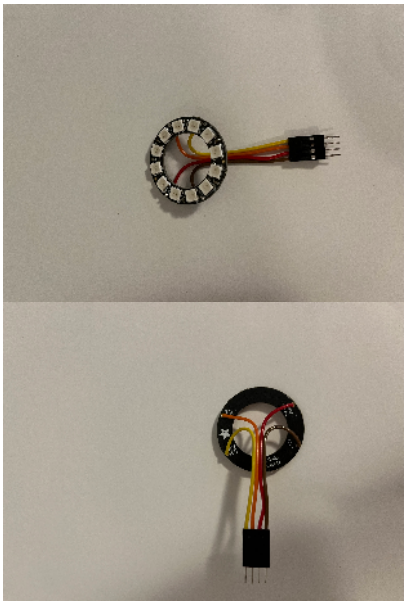
Das Hardware-Setup:

Für den Test und ersten Betrieb der Luftqualitätsampel haben wir folgendes Hardware-Setup verwendet:

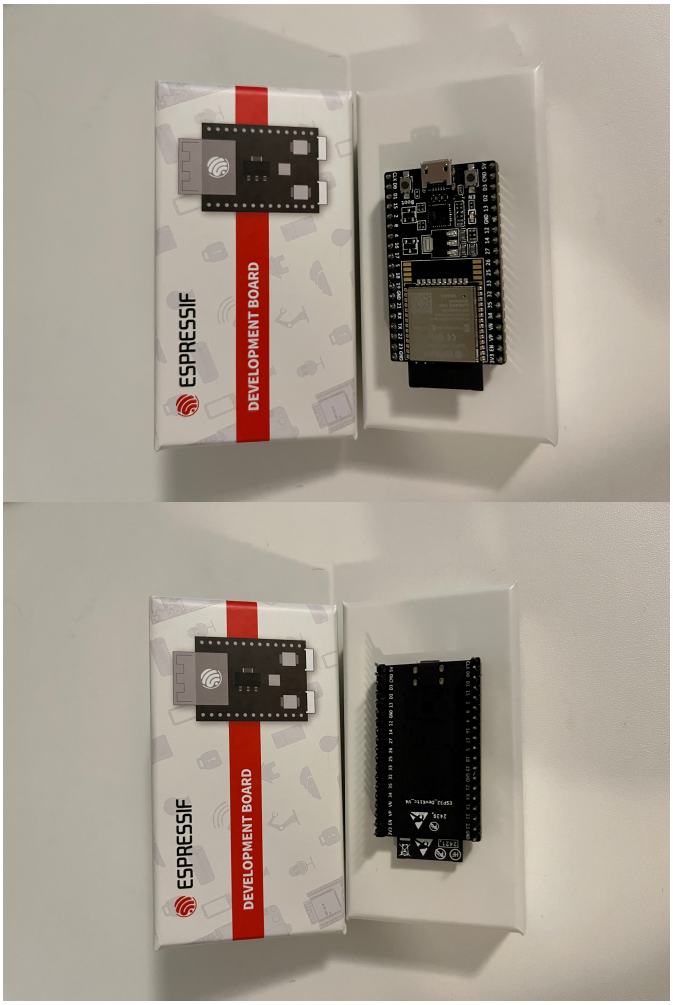


1. **Breadboard**

2. **NeoPixel-Ring** mit angelöteten Strom- und Datenkabeln:



3. **ESP32-DevKit**:



4. **Widerstand (330 Ω)** → in "h"-Lane des Breadboards (verbindet Data-In vom NeoPixel-Ring ["i"-Lane] mit DataPort "5" vom ESP32 ["g"-Lane]

5. **Micro-USB-Stromversorgung** des ESP32

6. **Jumper-Kabel** vom NeoPixel-Ring zum Breadboard:
   grün → 5V Stromversorgung für NeoPixel-Ring via Jumper-Kabel zur Power-Rail im Breadboard ⊕
   blau → Ground/Masse für den NeoPixel-Ring via Jumper-Kabel zur Power-Rail im Breadboard ⊖
   gelb → Data-In-Port vom NeoPixel-Ring via Jumper-Kabel zur "i"-Lane des Breadboards

7. **Jumper-Kabel** zur Verbindung des Data-Ports "5" vom ESP32 mit der "g"-Lane des Breadboards

8. **Jumper-Kabel** vom 5V-Port des ESP32 zur Power-Rail ⊕ des Breadboards um dort Strom zum Betrieb des NeoPixel-Rings bereitzustellen

9. **Jumper-Kabel** vom GND-Port des ESP32 zur Power-Rail ⊖ des Breadboards für Ground/Masse

> **Hinweis:**
> Für den Realbetrieb ist geplant unser bisheriges Test-Setup durch eine fest-verkabelte Variante zu ersetzen. Leider gibt es zum aktuellen Zeitpunkt jedoch

> noch Probleme mit der Lieferung der dafür benötigten Hardware, weshalb noch keine Bilder davon in dieser Dokumentation eingefügt werden können. Ebenso ist der Einbau der Ampel in ein 3D-gedrucktes Gehäuse geplant.
> Bei der Projektvorstellung im Rahmen der Programmierprojekt-Messe am 30.07.2025 wird diese Variante mit Gehäuse voraussichtlich fertig gebaut sein und wird dort zur Live-Demonstration verwendet. Es sei jedoch angemerkt, dass auch mit dem oben gezeigten bisherigen Prototyp mit dem Breadboard der korrekte Realbetrieb möglich wäre.

Weitere nötige vorbereitende Maßnahmen am Raspberry Pi (oder zum Test parallel am Laptop/PC):

Um **Mosquitto**, den MQTT-Broker den wir verwenden, auf einem Raspberry Pi (oder Laptop) zu installieren müssen folgende Schritte vorgenommen werden:

1. **System aktualisieren**
   ```
   sudo apt update
   sudo apt upgrade -y
   ```

2. **Mosquitto und Clients installieren**
   `sudo apt install -y mosquitto mosquito-clients`
3. **Mosquitto als Dienst aktivieren (optional)**
   `sudo systemctl enable mosquitto`
4. **Dienst starten**
   `sudo systemctl start mosquitto`

Zum Testen des MQTT-Brokers können in zwei separaten Terminals folgende Befehle ausgeführt werden:

1. **Terminal (Subscriber):**
   ```
   mosquitto_sub -h localhost -t test/topic
   ```
2. **Terminal (Publisher):**
   ```
   mosquitto_pub -h localhost -t test/topic -m "Hallo MQTT!"
   ```

   → **Jetzt sollte in Terminal 1 (Subscriber) "Hallo MQTT!" erscheinen und der Broker funktioniert korrekt**

MQTT-Setup Troubleshooting - CM

Troubleshooting am Raspberry Pi:

Wenn beim Start des Mosquitto-Brokers MQTT-Clients aus dem LAN beim Verbindungsversuch die Meldung „Connection refused“ angezeigt wird, liegt das häufig daran, dass der Dienst standardmäßig nur auf der Loopback-Schnittstelle (localhost) lauscht und anonyme Verbindungen unterbindet. Abhilfe schafft eine kurze Anpassung der Mosquitto-Konfiguration:

1. Öffnen Sie die Datei **/etc/mosquitto/mosquitto.conf** (bzw. in **/etc/mosquitto/conf.d/**) mit Root-Rechten.
2. Kommentieren Sie alle Zeilen mit **bind_address 127.0.0.1** oder **listener 1883** aus, damit keine doppelten Listener-Definitionen mehr existieren.

3. Fügen Sie an geeigneter Stelle (z. B. am Ende) folgende Direktiven ein:
   ```
   listener 1883
   allow_anonymous true
   ```

   → **listener 1883** sorgt dafür, dass Mosquitto auf **allen** Netzwerk-Interfaces (0.0.0.0) lauscht.

   → **allow_anonymous true** erlaubt Clients, sich ohne Benutzername und Passwort zu verbinden.

4. Speichern Sie die Datei und starten Sie den Dienst neu:
   ```
   sudo systemctl restart mosquitto
   ```

5. Überprüfen Sie mit:
   ```
   sudo lsof –i TCP:1883
   ```
   dass jetzt ***:1883 (LISTEN)** anstatt wie zuvor nur **localhost:1883 (LISTEN)** angezeigt wird.

Anschließend sollten mosquitto_pub- und mosquitto_sub-Befehle von jedem Rechner im gleichen Subnetz erfolgreich durchgehen, ohne dass die Verbindung zunächst abgelehnt wird.

PlatformIO als Build-Tool - CM

Für die ersten einfachen Betriebstests der LEDs haben wir relativ simple Arduino-Sketches verwendet, da hiermit sehr schnell und einfach die Grundlegende Funktionalität der LEDs getestet werden kann.
Während die Arduino-IDE für solche ersten Prototyping-Schritte hervorragend geeignet ist, stößt man bei einem wachsenden Projekt wie unserem allerdings schnell an ihre Grenzen, da die Bibliotheksverwaltung manuell erfolgt, Projektdaten oft in einzelnen Sketch-Dateien liegen, und Debug- sowie Build-Optionen stark eingeschränkt sind.

Mit **PlatformIO** als Erweiterung in VS Code gewannen wir hingegen eine professionelle, modulare Entwicklungsumgebung, die speziell für Embedded-Projekte konzipiert ist:

- **Bibliotheks- und Abhängigkeitsmanagement:** Library-Versionen (z. B. ArduinoJson, PubSubClient, Adafruit NeoPixel) werden zentral in platformio.ini deklariert und bei Bedarf automatisch heruntergeladen. Das verhindert Versionskonflikte und vereinfacht das Onboarding neuer Entwickler.
- **Multi-File-Projektstruktur:** Statt eines monolithischen Sketches erlaubt PlatformIO eine übersichtliche Trennung in Header /C++-Quellen (.h/.cpp), Tests und Konfigurationsdateien. So lässt sich unser Fade-Helper, der MQTT-Manager oder die LED-Controller-Klasse klar voneinander abgrenzen und wiederverwenden.
- **Integrierte Debugging-Unterstützung:** Über GDB und die serielle Konsole können Breakpoints gesetzt, Variablen live inspiziert und serielle Logausgaben direkt im Editor verfolgt werden. Das beschleunigt die Fehlersuche in Schleifen wie handleFade() oder beim MQTT-Reconnect erheblich.
- **Custom Build-Flags und Umgebungen:** Unterschiedliche Build-Profile (z. B. Debug versus Release, alternative WLAN-Konfigurationen) lassen sich unkompliziert per env:-Sektionen im platformio.ini anlegen. Damit ist es möglich, ohne Codeänderungen etwa Test- und Produktiv-Firmware zu generieren.
- **Task-Integration und Continuous Integration:** PlatformIO bietet CLI-Befehle für automatisierte Builds und Unit-Tests, lässt sich einfach in CI/CD-Pipelines einbinden und liefert konsistente Ergebnisse auf allen Plattformen (Windows, macOS, Linux).

Durch den Wechsel von der Arduino-IDE zu PlatformIO profitiert unser Projekt also nicht nur von einer saubereren Code-Organisation, sondern es wird auch die Grundlage für langfristige Wartbarkeit, reproduzierbare Builds und effizientes Team-Development gelegt.

Code für den ESP32 - CM

Alles erweitern Alles ausblenden

Übersicht der Code-Komponenten - CM

Datei	Zweck
config.h	Zentrale Definition von WLAN-, MQTT- und NeoPixel-Parametern
FadeHelper.h/.cpp	Interpolationslogik für sanfte Farbverläufe („Fading“)
LedController.h/.cpp	Steuerung des NeoPixel-Rings: Ein-/Aus-Färbung, Fading, Blinken
MqttManager.h/.cpp	Aufbau und Pflege der MQTT-Verbindung, Topic-Subscription
WiFiManager.h/.cpp	Aufbau und Überwachung der WLAN-Verbindung
main.cpp	Initialisierung, Callback-Handling, Hauptschleife (Loop)
platform.ini	Build- und Abhängigkeits-konfiguration für PlatformIO

Wichtige Code-Bestandteile im Detail - CM

config.h

Definiert als einzige Datei alle projektweiten Konstanten.

```
// WLAN
static constexpr char WIFI_SSID[]     = "ssid";
static constexpr char WIFI_PASSWORD[] = "password";

// MQTT
static constexpr char MQTT_SERVER[]   = "172.20.10.4";
static constexpr uint16_t MQTT_PORT   = 1883;
static constexpr char MQTT_TOPIC[]    = "airquality/command/led";

// NeoPixel
static constexpr uint8_t  LED_PIN     = 5;
static constexpr uint16_t NUMPIXELS   = 12;

// Fading
static constexpr uint16_t FADE_STEPS_DEFAULT    = 50;
static constexpr uint16_t FADE_INTERVAL_DEFAULT = 20; // ms
```

> **Erklärung:**
> Diese Datei zentralisiert alle hardware- und netzwerkbezogenen Konstanten. Änderungen – etwa an SSID, MQTT-Serveradresse oder LED-Anzahl – werden hier vorgenommen, ohne den übrigen Code anzupassen.

FadeHelper

Stellt eine **interpolate()**-Funktion bereit, die zwei RGB-Werte über eine gegebene Schrittzahl linear überblendet.

```
inline RGB interpolate(const RGB& start, const RGB& target, uint16_t step, uint16_t maxSteps) {
  if (maxSteps == 0) return start;
  float frac = float(step) / maxSteps;
  return {
    uint8_t(start.r + (target.r - start.r) * frac),
    uint8_t(start.g + (target.g - start.g) * frac),
    uint8_t(start.b + (target.b - start.b) * frac)
  };
}
```

> **Erklärung:**
> Die Funktion berechnet einen Zwischenfarbwert auf einer geraden Linie zwischen start und target nach dem Verhältnis step/maxSteps. So entsteht ein sanfter Farbverlauf.

LedController

Kapselt alle LED-Operationen inklusive sanfter Farbwechsel und Blink-Modus.

LedController::begin() & Konstruktor

```
LedController::LedController(uint8_t pin, uint16_t numPixels)
 : strip(numPixels, pin, NEO_GRB + NEO_KHZ800),
   _curR(0), _curG(0), _curB(0),
   _fadeSteps(FADE_STEPS_DEFAULT),
   _fadeInterval(FADE_INTERVAL_DEFAULT),
   _currentStep(0), _fading(false),
   _blinking(false), _blinkInterval(0), _lastBlinkTime(0), _blinkState(false)
{}

void LedController::begin() {
  strip.begin();
  strip.show(); // Alle Pixel zunächst auf „aus“
}
```

> **Erklärung:**
> Der Konstruktor legt den NeoPixel-Streifen an Pin 5 mit 12 LEDs an und initialisiert alle Status-Variablen. begin() startet die Bibliothek und stellt sicher, dass zu Beginn keine LEDs leuchten.

setColor(...)

```
void LedController::setColor(uint8_t r, uint8_t g, uint8_t b) {
  _prevR = r; _prevG = g; _prevB = b;
  _curR = r; _curG = g; _curB = b;
  for (uint16_t i = 0; i < strip.numPixels(); i++)
    strip.setPixelColor(i, strip.Color(r, g, b));
  strip.show();
}
```

> **Erklärung:**
> Schaltet alle LEDs sofort auf die übergebenen RGB-Werte. Speichert die aktuelle und die zuletzt dauerhaft gesetzte Farbe in den Member-Variablen _curR/G/B und _prevR/G/B.

setColorSmooth(...)

→ Initialisiert ein Fading-Intervall:

```
void LedController::setColorSmooth(uint8_t r, uint8_t g, uint8_t b, uint16_t steps, uint16_t inte
  stopBlinking();
  _prevR = r; _prevG = g; _prevB = b;
  _startR = _curR; _startG = _curG; _startB = _curB;
  _targetR = r;    _targetG = g;    _targetB = b;
  _fadeSteps = steps;
  _fadeInterval = interval;
  _currentStep = 0;
  _lastFadeTime = millis();
  _fading = true;
}
```

> **Erklärung:**
> Leitet ein sanftes Überblenden von der aktuellen Farbe (_curR/G/B) zur Zielfarbe (r/g/b) ein. steps und interval definieren die Dauer und Geschwindigkeit des Fadings.

blink(...)

→ Toggle-Logik für Warnungen:

```
void LedController::blink(uint8_t r, uint8_t g, uint8_t b, uint32_t intervalMs) {
  stopBlinking();
  _blinkColor = { r, g, b };
  _blinkInterval = intervalMs;
  _lastBlinkTime = millis();
  _blinkState = false;
  _blinking = true;
}
```

> **Erklärung:**
> Aktiviert den Blinkmodus: Die LEDs wechseln in der angegebenen Farbe r/g/b im Intervall intervalMs zwischen an und aus.

handleFade()

→ Muss im Hauptloop aufgerufen werden und Entscheidet zwischen Blinken und Fading:

```
void LedController::handleFade() {
  uint32_t now = millis();

  // 1) Wenn Blinken aktiv: Blinken primär abarbeiten
  if (_blinking) {
    if (now - _lastBlinkTime >= _blinkInterval) {
      _lastBlinkTime = now;
      _blinkState = !_blinkState;
      for (uint16_t i = 0; i < strip.numPixels(); i++) {
        if (_blinkState)
          strip.setPixelColor(i, strip.Color(_blinkColor.r, _blinkColor.g, _blinkColor.b));
        else
          strip.setPixelColor(i, strip.Color(0, 0, 0));
      }
      strip.show();
    }
    return;
  }

  // 2) Sonst: Fading Schritt ausführen, wenn fällig
  if (!_fading) return;
  if (now - _lastFadeTime < _fadeInterval) return;
  _lastFadeTime = now;
  if (++_currentStep > _fadeSteps) {
    setColor(_targetR, _targetG, _targetB);
    _fading = false;
    return;
  }

  RGB start  = { _startR, _startG, _startB };
  RGB target = { _targetR, _targetG, _targetB };
  RGB cur    = interpolate(start, target, _currentStep, _fadeSteps);

  setColor(cur.r, cur.g, cur.b);
}
```

> **Erklärung:**
> Wird in jeder Loop-Iteration aufgerufen und entscheidet:
>
> 1. Ob gerade geblinkt werden soll (bei erhöhter Gefahr)
> 2. Ob ein Fading-Schritt fällig ist → führt diesen per interpolate() aus.

MqttManager

Vereinfacht die Verbindung zum MQTT-Broker und leitet eingehende Nachrichten an eine Callback-Funktion weiter.

MqttManager::generateClientId()

```
void MqttManager::generateClientId() {
  uint64_t chipid = ESP.getEfuseMac();
  char buf[32];
  snprintf(buf, sizeof(buf), "ESP32_%04X%08X", (uint16_t)(chipid>>32), (uint32_t)chipid);
  _clientId = String(buf);
}
```

> **Erklärung:**
> Erzeugt eine eindeutige Client-ID für die MQTT-Verbindung.
> Dazu wird die weltweit einzigartige MAC-Adresse des ESP32-Chips verwendet und als Zeichenkette im Format "ESP32_<ChipID>" formatiert.
> So wird sichergestellt, dass auch wenn mehrere Geräte gleichzeitig betrieben werden, diese sich nicht gegenseitig aus dem MQTT-Netzwerk verdrängen.

MqttManager::begin(...)

```
void MqttManager::begin(const char* server, uint16_t port) {
  generateClientId();
  _client.setServer(server, port);
}
```

> **Erklärung:**
> Initialisiert die MQTT-Verbindung mit der angegebenen Server-Adresse und Portnummer.
> Vor dem Setzen der Verbindungsdaten wird einmalig die eindeutige Client-ID generiert.
> Damit ist das Gerät korrekt im MQTT-Broker registriert und bereit für alle folgenden Verbindung- und Anmeldevorgänge.

MqttManager::setCommandCallback(...)

```
void MqttManager::setCommandCallback(std::function<void(const char*, const uint8_t*, unsigned int
  _client.setCallback([cb](char* topic, byte* payload, unsigned int length){
    cb(topic, payload, length);
  });
}
```

> **Erklärung:**
> Registriert die vom Anwender übergebene Callback-Funktion (cb), die ausgeführt wird, wenn eine neue MQTT-Nachricht eintrifft. Da PubSubClient eine anders geformte Signatur für seinen Callback erwartet, wird hier ein Lambda benutzt, das die Parameter einfach an cb weiterleitet.

MqttManager::reconnect()

→ Exponentielles Back-off bis zur erfolgreichen Verbindung:

```
void MqttManager::reconnect() {
  if (_client.connected()) return;
  unsigned long now = millis();
  if (now - _lastReconnectAttempt < _reconnectDelay) return;
  _lastReconnectAttempt = now;
  Serial.printf("MQTT connect (%s), delay %lums\n", _clientId.c_str(), _reconnectDelay);
  if (_client.connect(_clientId.c_str())) {
    _client.subscribe(MQTT_TOPIC);
    Serial.println("MQTT verbunden");
    _reconnectDelay = 2000;
  } else {
    Serial.println("MQTT Verbindungsfehler.");
    _reconnectDelay = min(_reconnectDelay * 2, (unsigned long)60000);
  }
}
```

> **Erklärung:**
> Sorgt dafür, dass das Gerät dauerhaft mit dem MQTT-Broker verbunden bleibt. Es wird in jedem Loop-Durchlauf geprüft, ob eine Verbindung besteht.
>
> - **Erfolgreicher Connect:** Sobald die Verbindung steht, wird das gewünschte Topic abonniert und das Initial-Delay zurückgesetzt.
> - **Fehlversuch + Back-off:** Bei Verbindungsproblemen wird der nächste Verbindungsversuch um einen exponentiellen Zeitraum verzögert (maximal 60 Sekunden), sodass das System weiterhin auf andere Aufgaben reagieren kann und nicht blockiert.

WiFiManager.cpp

Kapselt Aufbau und Überwachung der WLAN-Verbindung

```
unsigned long WiFiManager::_lastAttempt = 0;
```

```
void WiFiManager::begin(const char* ssid, const char* password) {
  WiFi.begin(ssid, password);
  Serial.print("WLAN verbindet");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print('.');
  }
  Serial.println("\nWLAN verbunden!");
}

void WiFiManager::maintain() {
  if (WiFi.status() != WL_CONNECTED) {
    unsigned long now = millis();
    if (now - _lastAttempt > 5000) { // Alle 5s versuchen
      Serial.println("WLAN getrennt, reconnect...");
      WiFi.reconnect();
      _lastAttempt = now;
    }
  }
}
```

> **Erklärung**
>
> Die Klasse übernimmt das Initialisieren der WLAN-Verbindung anhand der in *config.h* hinterlegen SSID und Passwort.
> Im Setup baut *begin()* die Verbindung auf und blockiert bis zum erfolgreichen Verbindungsaufbau.
> Die Methode *maintain()* überprüft in jeder Loop-Iteration, ob die Verbindung noch steht und stößt im Fehlerfall alle 5 Sekunden einen Reconnect an, um unnötige Wiederholungen zu vermeiden und das System zu entlasten.

main.cpp

Orchestriert Setup, Loop und die Übersetzung eingehender MQTT-Nachrichten in LED-Aktionen.

AirQualityLevel-Enum & Parser

```
enum class AirQualityLevel {
  SehrGut,
  Gut,
  Maessig,
  Schlecht,
  SehrSchlecht,
  Unbekannt
};

AirQualityLevel parseAirQuality(const String& msg) {
  String m = msg;
  m.toLowerCase();
  m.trim();
  if (m == "sehr gut") return AirQualityLevel::SehrGut;
  if (m == "gut")      return AirQualityLevel::Gut;
  if (m == "maessig")  return AirQualityLevel::Maessig;
  if (m == "schlecht") return AirQualityLevel::Schlecht;
```

```
  if (m == "sehr schlecht") return AirQualityLevel::SehrSchlecht;
  return AirQualityLevel::Unbekannt;
}
```

> **Erklärung:**
>
> - Das **Enum** definiert alle möglichen Luftgüte-Stufen, die das System kennt.
> - Die **Parser-Funktion** wandelt eine empfangene Textnachricht in einen dieser Enum-Werte um.
> - Durch **toLowerCase()** und **trim()** wird das Message-Format robust gegen Groß-/Kleinschreibung und überflüssige Leerzeichen gemacht.
> - Treffer in den **if**-Blöcken ordnen das Ergebnis der passenden Stufe zu → unbekannte oder fehlerhafte Nachrichten landen in **Unbekannt**.

MQTT-Callback

```
void onMqttMessage(const char* topic, const uint8_t* payload, unsigned int length) {
  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, payload, length);
  if (!err && doc.containsKey("r") && doc.containsKey("g") && doc.containsKey("b")) {
    int r = doc["r"];
    int g = doc["g"];
    int b = doc["b"];
    uint16_t steps   = doc["steps"]   | FADE_STEPS_DEFAULT;
    uint16_t delayMs = doc["delayMs"] | FADE_INTERVAL_DEFAULT;
    leds.setColorSmooth(safeColor(r), safeColor(g), safeColor(b), steps, delayMs);
    return;
  }

  // Ansonsten: reiner Text-Messagehandler
  String msg;
  for (unsigned int i = 0; i < length; i++) msg += (char)payload[i];
  msg.trim();

  AirQualityLevel level = parseAirQuality(msg);
  switch (level) {
    case AirQualityLevel::SehrGut:
      leds.stopBlinking();
      leds.setColorSmooth(0, 255, 0);
      break;
    case AirQualityLevel::Gut:
      leds.stopBlinking();
      leds.setColorSmooth(127, 255, 0);
      break;
    case AirQualityLevel::Maessig:
      leds.stopBlinking();
      leds.setColorSmooth(255, 165, 0);
      break;
    case AirQualityLevel::Schlecht:
      leds.stopBlinking();
      leds.setColorSmooth(200, 0, 0);
      break;
    case AirQualityLevel::SehrSchlecht:
      leds.blink(255, 0, 0, 250);
      break;
    default:
      leds.stopBlinking();
      leds.setColorSmooth(128, 128, 128);
```

```
      break;
  }
}
```

> **Erklärung:**
> Reagiert auf eingehende MQTT-Nachrichten. Entweder wird ein JSON-Kommando (r/g/b, steps, delayMs) ausgeführt oder eine einfache Textnachricht in eine Luftgüte-Stufe übersetzt, um die Ampelfarbe bzw. Blinkmodus zu setzen.

Setup

```
void setup() {
  Serial.begin(115200);
  leds.begin();
  WiFiManager::begin(WIFI_SSID, WIFI_PASSWORD);
  mqttMgr.begin(MQTT_SERVER, MQTT_PORT);
  mqttMgr.setCommandCallback(onMqttMessage);
}
```

> **Erklärung:**
>
> 1. **Serial.begin(115200)** → Initialisiert die serielle Schnittstelle zur Ausgabe von Debug- und Statusmeldungen im Monitor.
> 2. **leds.begin()** → Startet den NeoPixel-Ring und löscht alle LEDs (sichert einen definierten Ausgangszustand).
> 3. **WiFiManager::begin(...)** → Baut die WLAN-Verbindung mithilfe der definierten SSID und des Passworts auf und blockiert bis zum Erfolg.
> 4. **mqttMgr.begin(...)** → Konfiguriert die Adresse und den Port des MQTT-Brokers.
> 5. **mqttMgr.setCommandCallback(onMqttMessage)** → Registriert die Funktion, die eingehende MQTT-Nachrichten verarbeitet und in LED-Befehle übersetzt

```
void loop() {
  leds.handleFade();
  WiFiManager::maintain();
  mqttMgr.loop();
}
```

> **Erklärung:**
>
> 1. **leds.handleFade()** → Führt in jeder Iteration entweder einen Blink- oder Fade-Schritt aus, je nachdem, welcher Modus aktiv ist.
> 2. **WiFiManager::maintain()** → Überprüft den WLAN-Status und verbindet bei einem Abbruch automatisch neu.

3. **mqttMgr.loop()** → Hält die MQTT-Verbindung am Leben, verarbeitet eingehende Nachrichten und löst bei Bedarf den Reconnect aus.

platform.ini

Enthält alle zentralen Build- und Umgebungsoptionen für das PlatformIO-Projekt.

→ Definiert das verwendete Board, das ESP32-Framework, die genutzten Bibliotheken sowie die serielle Monitor-Geschwindigkeit.

```
[env:rymcu-esp32-devkitc]
platform = espressif32
board = rymcu-esp32-devkitc
framework = arduino
monitor_speed = 115200
lib_deps =
    bblanchon/ArduinoJson@^6.20.0
    adafruit/Adafruit NeoPixel@^1.10.0
    knolleary/PubSubClient@^2.8
```

Erklärung:

- **platform**: Gibt die Plattform (*espressif32* für ESP32) an.
- **board**: Wählt das konkrete Board-Modell (hier *rymcu-esp32-devkitc*).
- **framework**: Nutzt das Arduino-Framework für einfaches ESP32-Programming.
- **monitor_speed**: Stellt die Geschwindigkeit des seriellen Monitors ein (115200 Baud).
- **lib_deps**: Listet alle verwendeten Bibliotheken samt Versionsnummern, die bei jedem Build automatisch installiert werden:
  - *ArduinoJson* für das Parsen von JSON-Nachrichten.
  - *Adafruit NeoPixel* zur Ansteuerung des LED-Rings.
  - *PubSubClient* für MQTT-Kommunikation.

Damit ist sichergestellt, dass das Projekt auf jedem Rechner und in jeder Entwicklungsumgebung **reproduzierbar** gebaut werden kann.

Teststrategie und Testausführung - CM

Ziel der Tests

Im Projekt werden Unit-Tests eingesetzt, um zentrale Anwendungslogik wie AirQuality-Parsing, Farbberechnung und andere reine Software-Funktionen unabhängig von der Hardware automatisiert zu prüfen. Dadurch können Fehler frühzeitig erkannt und die Wartbarkeit des Projekts verbessert werden.

Testansatz

Es wird zwischen zwei Kategorien von Code unterschieden:

1. Logikfunktionen ohne Hardwarebezug

- **Beispiele:** AirQuality-Parsing, Farbberechnungen, Interpolationen
- **Test:** Diese Funktionen sind unabhängig von der ESP32-Hardware und werden direkt auf dem PC getestet (“Host Testing”).

2. Hardwareabhängige Komponenten

- **Beispiele:** Ansteuerung des NeoPixel-Rings, MQTT-Kommunikation, WLAN-Handling
  Test: Für diese Bereiche sind klassische Unit-Tests nur eingeschränkt sinnvoll, da sie Hardwarezugriffe erfordern. Sie werden
- daher durch gezielte manuelle Tests am Gerät überprüft.

Konkrete Teststrategie

- **Alle Unit-Tests** werden als eigenständiges, plattformunabhängiges C++-Testfile entwickelt und auf dem PC ausgeführt.
- **Catch2** wird als Framework für einfache, leicht lesbare Unit-Tests verwendet. Die Tests sind unabhängig von Arduino- oder ESP32-spezifischen Bibliotheken und laufen schnell lokal oder in der CI/CD.
- Die Testfälle werden in einer Datei (z.B. test_logic.cpp) zentral verwaltet.
- Das Testfile enthält umfassende Tests für alle Kernfunktionen der Logik, z.B.:
  - **parseAirQuality:** Robustheit gegenüber Groß-/Kleinschreibung, Whitespace, ungültigen Eingaben.
  - **safeColor:** Korrekte Begrenzung von Farbwerten außerhalb des erlaubten Bereichs.
  - **interpolate:** Richtiges Interpolieren von Farbwerten bei unterschiedlichen Start- und Zielwerten.

Begründung für diesen Ansatz

- **Schnelle Ausführung:** Tests laufen in Sekundenbruchteilen auf dem PC, unabhängig von Embedded-Hardware.
- **Einfache Integration:** Alle Teammitglieder können die Tests ausführen, ohne spezielle Hardware zu benötigen.
- **Klares Feedback:** Fehler in der Logik werden sofort erkannt, Refactorings sind risikofrei möglich.
- **Portabilität:** Die Teststrategie ist unabhängig vom Betriebssystem oder der späteren Hardware.

Ergänzende Testmaßnahmen

Die Überprüfung der Hardware-Funktionen (z.B. LED-Ansteuerung, MQTT-Verbindung) erfolgt durch gezielte manuelle Tests am ESP32, wie z.B. durch Senden von Farbbefehlen und Beobachtung der tatsächlichen LED-Reaktionen.

Beispielhafter Testablauf

1. **Code-Änderung** an einer Logikfunktion (z.B. AirQuality-Parsing).
2. **Ausführung der Unit-Tests** am PC (./test_logic).
3. **Feedback**: Es wird sofort angezeigt, ob alle Tests bestanden wurden.
4. **Integration**: Anschließend erfolgt die Integration auf dem ESP32 und die manuelle Prüfung der Hardwarefunktionen.

Anleitung: Ausführung der Unit-Tests auf dem PC

Voraussetzungen

- **C++ Compiler** (z.B. g++, clang, MSVC)
- **Catch2 Test-Framework** (nur .hpp nötig) → Ist beim Klonen unseres Repositories bereits im ESP32-Test-Directory enthalten

Schritte:

1. **Catch2 herunterladen (falls nicht schon vorhanden)**
   Die aktuelle Catch2 Single-Header-Datei hier herunterladen und im selben Verzeichnis wie die Testdatei (test_logic.cpp) absichern.

2. **Testdatei ablegen**
   Die Datei *test_logic.cpp* ins gleiche Verzeichnis.

3. **Kompilieren**
   ```
   g++ -std=c++17 -o test_logic test_logic.cpp
   ```
   (Alternativ: clang++ -std=c++17 -o test_logic test_logic.cpp)

4. **Tests ausführen**
   `./test_logic`
5. **Ergebnis**
   Es erscheint eine Übersicht aller Testfälle mit Angabe von "passed"/"failed".

> **Hinweis:**
>
> Falls der Fehler *'catch.hpp' file not found* auftritt, muss erneut überprüft und sichergestellt werden, dass sich die Datei catch.hpp im gleichen Verzeichnis wie test_logic.cpp befindet und im Testfile der Include mit *#include "catch.hpp"* erfolgt.

MQTT-Publishing vom Raspberry Pi - CM

Ziel

Das Ziel war, die bestehende API (api.py, FastAPI-basiert) so zu erweitern, dass kontinuierlich das aktuelle Luftqualitätslevel via MQTT gepublisht wird. Um Robustheit zu gewährleisten, sollte die API zudem automatisch versuchen, eine verlorene MQTT-Verbindung wiederherzustellen.

Umsetzung

Die Implementierung erfolgte direkt in der vorhandenen api.py und umfasst folgende Aspekte:

1. Initialisierung des MQTT-Clients

Beim Start der API wird der MQTT-Client initialisiert und verbindet sich mit dem lokalen MQTT-Broker:

```
@app.on_event("startup")
async def startup_event():
    global mqtt_client

    try:
        mqtt_client = MQTTClient(MQTT_BROKER, MQTT_PORT, MQTT_TOPIC)
        print(f"MQTT Client verbunden mit {MQTT_BROKER}:{MQTT_PORT}, Topic: {MQTT_TOPIC}")
    except Exception as e:
        print(f"MQTT Client konnte nicht verbunden werden: {e}")
        mqtt_client = None

    asyncio.create_task(live_data_loop())
    asyncio.create_task(mqtt_reconnect_loop())
```

2. Kontinuierliches Publishing der Luftqualität

Im bestehenden live_data_loop() werden regelmäßig Sensordaten verarbeitet und das ermittelte Luftqualitätslevel per MQTT veröffentlicht:

```
async def live_data_loop():
    global mqtt_client

    while True:
        try:
            raw_line = sensor_reader.lese_daten()
            if not raw_line:
                await asyncio.sleep(1)
                continue

            rating_vals = sensor_reader.get_rating_measurements(raw_line)
            rules = get_air_quality_rules()
            overall_quality = rules.get_overall_air_quality(rating_vals)
            overall_quality_str = air_quality_to_string(overall_quality)
```

```
            if mqtt_client:
                try:
                    mqtt_client.publish(overall_quality_str)
                except Exception as e:
                    print(f"Fehler beim MQTT Publish: {e} - MQTT Client wird neu verbunden")
                    mqtt_client = None  # Löst reconnect aus

        except Exception as e:
            print(f"Error in live data loop: {e}")

        await asyncio.sleep(1)
```

3. Automatische Wiederverbindung zum MQTT-Broker

Der Task mqtt_reconnect_loop() sorgt dafür, dass der MQTT-Client, falls die Verbindung verloren geht, automatisch alle 30 Sekunden versucht, sich erneut mit dem Broker zu verbinden:

```
async def mqtt_reconnect_loop():
    global mqtt_client

    while True:
        if mqtt_client is None:
            try:
                mqtt_client = MQTTClient(MQTT_BROKER, MQTT_PORT, MQTT_TOPIC)
                print(f"[MQTT] Reconnected to broker at {MQTT_BROKER}:{MQTT_PORT}")
            except Exception as e:
                print(f"[MQTT] Reconnect failed: {e}")
        await asyncio.sleep(30)
```

Funktionsweise und Vorteile

- **Robustheit**: Bei temporären Verbindungsproblemen oder einem Neustart des Brokers baut die API die Verbindung automatisch wieder auf.
- **Performance**: Die asynchronen Tasks beeinflussen die Performance der restlichen API-Endpunkte nicht spürbar.
- **Zuverlässigkeit**: Externe Systeme (z. B. LED-Indikatoren oder Dashboards), die MQTT-Nachrichten abonnieren, erhalten kontinuierlich aktuelle Luftqualitätsdaten, auch bei kurzen Unterbrechungen.

Fazit

Durch diese Integration wird eine stabile und wartungsarme Bereitstellung aktueller Luftqualitätsdaten über MQTT ermöglicht, ohne den laufenden Betrieb der API negativ zu beeinflussen.

Sonstiges

Versionsverwaltung & Projektstruktur - PP

1. Projekt-Repository und Git-Workflows

- **Repository-Klonen**
  Das zentrale Repository wurde mit folgendem Befehl geklont:

  ```
  git clone <repo-url>
  ```

  (URL ggf. durch Bitbucket-URL ersetzen.)
- **Alltägliche Git-Kommandos**
  - Änderungen werden mit `git add` zur Staging-Area hinzugefügt.
  - Mit `git commit -m "Commit-Message"` werden Änderungen lokal gespeichert.
  - Über `git push` werden die Commits ins Remote-Repo übertragen.
  - Um den aktuellen Stand vom Server zu holen, wird `git pull` verwendet.
- **.gitignore**
  Die Datei `.gitignore` enthält Regeln, um temporäre Dateien, virtuelle Umgebungen und andere nicht relevante Daten aus dem Repo auszuschließen, z. B.:

  ```
  __pycache__/
  *.pyc
  *.db
  *.log
  venv/
  ```

---

2. Virtuelle Umgebung & Abhängigkeiten

- **Virtuelle Umgebung anlegen**
  Mit folgendem Befehl wurde die venv im Projektordner erstellt:
  `python -m venv venv`

  (Wichtig: venv liegt **innerhalb** des Repos, ist aber per `.gitignore` ausgeschlossen.)
- **Requirements-Datei**
  Nach jeder Paketinstallation mit pip wurde die `requirements.txt` durch
  `pip freeze > requirements.txt`

  aktualisiert. Sie enthält alle benötigten Python-Abhängigkeiten für die Reproduktion der venv.

---

3. Projektstruktur

Die Projektstruktur ist nach Themen unterteilt (Sensorik, Aktorik, Datenbank etc.):

```
gruppe-04---air-quality-monitoring:.
   .gitignore
   p100ansteuern.py
   package-lock.json
   projektstruktur.txt

 .vscode/
    settings.json

 airquality/
      api.py
      docker-compose.yml
      DOCKERFILE.backend
      DOCKERFILE.frontend
      ips7100_data.log
      main.py
      mqtt_client.py
      pytest.ini
      requirements.txt
      tp100_controller.py

    aktorik/
         air_control_rules.py
         air_quality_rating.py
         __init__.py

    database/
         air_quality.db
         db_helper.py
         save_data.py
         __init__.py

    docs/
         README.md

    sensorik/
         read_sensor_data.py
         __init__.py

    tests/
          test_air_control_rules.py
          test_air_quality_rating.py
          test_db_helper.py
          test_read_sensor_data.py
          test_save_data.py
          __init__.py

 ESP32AirQuality/
      .gitignore
      platformio.ini

    .pio/
       build/
         project.checksum
       libdeps/
           ... (Libraries, Beispiele, Tests für ESP32)
    .vscode/
       ...
    include/
       ...
    lib/
       ...
    src/
       main.cpp
    test/
```

```
        ...

frontend/
       .env
       .env.test
       .gitignore
       babel.config.cjs
       DOCKERFILE.frontend
       index.html
       jest.config.cjs
       jest.setup.cjs
       package-lock.json
       package.json
       postcss.config.js
       README.md
       tailwind.config.js
       vite.config.js

     public/
        vite.svg
     src/
        api/
           api.js
        assets/
           react.svg
        components/
           AirQualitySummary.jsx
           DeviceControlPanel.jsx
           Header.jsx
           HistoryChart.jsx
           HistoryFilterBar.jsx
           ModeSwitch.jsx
           ParticleChartCard.jsx
           SensorMetricCard.jsx
           SessionManager.jsx
           ...
        hooks/
           useLiveStream.js
           useSessionDelta.js
        pages/
           Dashboard.jsx
        services/
           api.js
        utils/
            envShim.js
            exportSession.js
     tests/
        components/
           DeviceControlPanel.test.jsx
           SessionManager.test.jsx
        hooks/
           useLiveStream.test.jsx
           useSessionDelta.test.jsx
        msw/
           handlers.js
           server.js
        services/
            api.test.js
     __mocks__/
         fileMock.js
         styleMock.js
```

Hinweise:

- Die Unterordner `aktorik`, `sensorik`, `database`, `tests` sind thematisch sauber getrennt.
- Die venv liegt im Projektordner, ist aber für Git ausgeschlossen.
- Im Ordner `docs` befindet sich die Projektdokumentation als README.md.

- Im Ordner `tests` liegen alle Unittests.
- Log- und Datenbankdateien (`*.log`, `*.db`) sind in der `.gitignore` ausgenommen.

Das Team

Name	Kürzel (siehe Überschriften-Suffix)
Christian Model	CM
Pascal Putz	PP
Steffen Krutzsch	SK
Marvin Kraus	MK

Deployment

Containerisierung mit Docker Compose - PP

1. Übersicht

Das gesamte System läuft als Docker-Compose-Stack und besteht aus folgenden Komponenten:

- **Backend (API):**
  FastAPI (Python), Zugriff auf den Luftqualitätssensor via USB.
- **Frontend:**
  Modernes SPA-Framework (z. B. Vite/React/Vue), Build-Argumente erlauben flexible API-URL.
- **SQLite-Datenbank:**
  Persistente Speicherung im lokalen Host-Ordner, unabhängig von Containern.

---

2. docker-compose.yml (finales Setup)

```
version: "3.9"
services:
  api:
    build:
      context: .
      dockerfile: DOCKERFILE.backend
    container_name: airquality-api
    ports:
      - "8000:8000"
    volumes:
      - ./database:/app/database      # SQLite-DB bleibt erhalten!
    environment:
      - PYTHONUNBUFFERED=1
    restart: unless-stopped
    devices:
      - "dev/ttyUSB0:/dev/ttyUSB0"

  frontend:
    build:
      context: ../frontend
      dockerfile: DOCKERFILE.frontend
      args:
        VITE_USE_MOCK: ${VITE_USE_MOCK:-false}
        VITE_API_URL:   ${VITE_API_URL:-http://172.20.10.4:8000}
    container_name: airquality-frontend

    ports:
      - '3000:80'
    restart: unless-stopped
    depends_on:
      - api

    environment:
      - VITE_USE_MOCK=${VITE_USE_MOCK:-false}
      - VITE_API_URL=${VITE_API_URL:-http://172.20.10.4:8000}
```

---

3. Erläuterungen zu den wichtigsten Optionen

api-Service:

- **devices:**
  `/dev/ttyUSB0` wird in den Container gemountet, damit der Sensor auch im Container direkt angesprochen werden kann.
  volumes:
- Das lokale Verzeichnis `./database` speichert die SQLite-Datenbank persistent und unabhängig vom Container-Lebenszyklus.
- **ports/environment/restart:**
  Die API läuft auf Port 8000. Bei Fehlern/Neustarts wird der Container automatisch wieder gestartet.

frontend-Service:

- **build/context/dockerfile:**
  Baut das Frontend aus dem `../frontend`-Verzeichnis mit dem angegebenen Dockerfile.
  args/environment:
- Die Variable `VITE_API_URL` verweist **fest auf die interne API-IP** (`http://172.20.10.4:8000`).
  Damit ist die API von allen Geräten im selben Netzwerk aus erreichbar – wichtig für echte Mehrgeräte-Umgebungen!
- **depends_on:**
  Das Frontend wird erst gestartet, wenn die API verfügbar ist.

---

4. Start und Nutzung

Stack starten:

```
docker-compose up --build
```

- **API erreichbar:**
  http://172.20.10.4:8000
- **Frontend erreichbar:**
  http://localhost:3000 (bei Zugriff auf dem Host)

---

5. Besonderheiten und Tipps

- Die feste API-URL (`VITE_API_URL`) sorgt dafür, dass das Frontend im Container immer die richtige Adresse für das Backend kennt – sowohl im lokalen Netzwerk als auch für mobile/weitere Geräte.
- Der USB-Sensor funktioniert im Container, da `/dev/ttyUSB0` direkt eingebunden wird.
- Die Datenbank bleibt immer im lokalen Ordner und ist somit leicht für Backups oder externe Analysen zugänglich.

Dockerfile Backend - CM

Zweck und Kontext

Dieses Dockerfile dient zum Aufbau eines isolierten Containers für das **FastAPI-Backend** des Projekts *Airquality-Monitoring*. Es erstellt eine saubere, reproduzierbare Laufzeitumgebung mit Python 3.11.2, in der alle benötigten Abhängigkeiten installiert sind und das Backend über den ASGI-Server **Uvicorn** gestartet wird.

Dockerfile-Inhalt

```
FROM python:3.11.2

WORKDIR /app

COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Erläuterung der einzelnen Anweisungen

`FROM python:3.11.2`	Nutzt das offizielle Python-Basisimage in Version 3.11.2.
`WORKDIR /app`	Legt das Arbeitsverzeichnis innerhalb des Containers auf `/app` fest.
`COPY requirements.txt ./`	Kopiert die Datei mit Python-Abhängigkeiten (`requirements.txt`) ins Arbeitsverzeichnis.
`RUN pip install --no-cache-dir -r requirements.txt`	Installiert alle Python-Abhängigkeiten ohne Cache, was das Image schlank hält.
`COPY . .`	Überträgt den kompletten Backend-Code in den Container.
`EXPOSE 8000`	Macht Port 8000 des Containers für eingehende HTTP-Anfragen erreichbar.
`CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]`	Startet den Uvicorn-Server mit der Anwendung `api:app` auf allen Netzwerkschnittstellen.

Hinweise zur Wartbarkeit

- Bei Änderungen an den Abhängigkeiten immer die `requirements.txt` aktualisieren, um die Konsistenz des Containers zu gewährleisten.

- Für kleinere Images empfiehlt sich ggf. später ein Umstieg auf `python:3.11-slim`.

Zusammenfassung

Das Dockerfile sorgt für eine stabile, standardisierte Laufzeitumgebung, die speziell auf das Backend des Airquality-Monitoring-Projekts abgestimmt ist. Durch die klaren und einfachen Anweisungen bleibt das Image wartbar und gut lesbar.

Dockerfile Frontend - CM

Zweck und Kontext

Dieses Dockerfile baut das Frontend des Projekts **Airquality-Monitoring** als optimierte, statische Web-Applikation in einer isolierten Umgebung. Es nutzt einen zweistufigen Build-Prozess, um ein schlankes und performantes Produktions-Image bereitzustellen. Das finale Image läuft mit dem Webserver **NGINX**, welcher die fertigen statischen Dateien bereitstellt.

Dockerfile-Inhalt

```
# Build Stage
FROM node:18-alpine as build
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build

# Production Stage
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Erläuterung der einzelnen Anweisungen

`FROM node:18-alpine as build`	Nutzt ein leichtgewichtiges Node.js-Image (Alpine) in Version 18 als Build-Umgebung.
`WORKDIR /app`	Setzt das Arbeitsverzeichnis innerhalb des Containers auf `/app`.
`COPY . .`	Kopiert den gesamten Frontend-Quellcode in das Arbeitsverzeichnis.
`RUN npm install`	Installiert alle Abhängigkeiten aus `package.json`.
`RUN npm run build`	Erzeugt optimierte, statische Dateien für die Produktionsumgebung.
`FROM nginx:alpine`	Startet eine neue, schlanke Laufzeit-Umgebung basierend auf NGINX-Alpine.
`COPY --from=build /app/build /usr/share/nginx/html`	Kopiert die generierten statischen Dateien aus dem vorherigen Build-Schritt in das NGINX-Standardverzeichnis.
`EXPOSE 80`	Macht Port 80 für HTTP-Anfragen zugänglich.
`CMD ["nginx", "-g", "daemon off;"]`	Startet den NGINX-Webserver im Vordergrund (Standard für Container-Betrieb).

Hinweise zur Wartbarkeit

- Änderungen der Frontend-Abhängigkeiten müssen immer in der `package.json` definiert sein.

- Bei größeren Änderungen ist ein regelmäßiger Cleanup (`npm prune` oder Ähnliches) ratsam, um Abhängigkeiten aktuell und sicher zu halten.

Zusammenfassung

Das Dockerfile baut und betreibt das Frontend als schlanke, effiziente Web-Anwendung auf Basis von NGINX. Durch die Nutzung von Multi-Stage Builds wird das resultierende Image klein und ideal für den Einsatz in Produktionsumgebungen optimiert.

Abschluss und Erweiterungspotenzial - PP

Fazit

Das AirQuality-Projekt bietet eine robuste und lokal vollständig betreibbare Lösung für die **Erfassung, Bewertung und Visualisierung von Luftqualitätsdaten**.
Durch die Containerisierung mit Docker Compose und die konsequente Modularisierung (Sensorik, Datenbank, Bewertung, UI) ist das System nicht nur wartbar und ausbaufähig, sondern auch besonders datensouverän und unabhängig von externen Cloud-Diensten.

Das flexible Session-Konzept, die klare Trennung von Sensorik, Datenhaltung und Regelwerk sowie die offene Architektur (API, Frontend, Datenexport) ermöglichen vielfältige Einsatzmöglichkeiten – vom Raumluft-Monitoring über wissenschaftliche Experimente bis zu automatisierten Smart-Home-Lösungen.

Erweiterungspotenzial

Session-Management im Frontend:

- Das Frontend kann erweitert werden, um Sessions direkt aus der Benutzeroberfläche zu löschen, zu archivieren oder umzubenennen (inklusive Bestätigungsdialog und ggf. Undo-Option).

Barrierefreiheit & Nutzerfreundlichkeit:

- Features wie Darkmode, kontrastreiche Themes, größere Schriften und Screenreader-Unterstützung sorgen für eine barrierefreie Nutzung – insbesondere für Menschen mit Sehbeeinträchtigungen.

Flexible Konfiguration durch Umgebungsvariablen:

- Zugangsdaten, API-URLs und weitere netzwerk- oder sicherheitsrelevante Einstellungen sollten nicht mehr hardcodiert, sondern zentral über eine `.env`-Datei gepflegt und im Code/Container automatisch eingelesen werden.
  Das erleichtert die Wartung und den Betrieb in unterschiedlichen Umgebungen und erhöht die Sicherheit.

Erweiterung auf weitere Aktorik und Sensorik:

- Die Plattform kann einfach um neue Sensoren (z. B. Schall, Helligkeit, weitere Luftgüteparameter) oder Aktoren (Luftreiniger, Fensteröffner, Heizungssteuerung etc.) ergänzt werden.

Alarm- und Benachrichtigungssystem:

- Bei Überschreitung von Schwellenwerten könnten automatische Warnungen per E-Mail, Push-Nachricht, Telegram oder andere Kanäle versendet werden.

Benutzerverwaltung:

- Für größere Teams oder öffentliche Installationen kann eine Benutzerverwaltung mit verschiedenen Rollen (Admin, Gast, User) ergänzt werden.

Mehrsprachigkeit:

- Die Oberfläche kann für die Nutzung in mehreren Sprachen (z. B. Deutsch/Englisch) vorbereitet werden.

Integration externer Datenquellen:

- Ein Abgleich der eigenen Messwerte mit offiziellen Wetter- oder Umweltwerten (z. B. vom Umweltbundesamt) könnte zusätzliche Informationen liefern.

Automatisierte Datensicherung:

- Automatische Backups der SQLite-Datenbank, z. B. per Cronjob oder als zusätzlicher Service im Docker Compose-Stack.

Lessons Learned

- Eine klare Modulstruktur und Docker-Containerisierung erleichtern die Entwicklung, das Testing und den Rollout enorm.
- Das sessionbasierte Datenmodell ist für praxisnahe Vergleiche und Auswertungen viel flexibler als eine rein laufende Messwertsammlung.
- Durch die konsequente Nutzung von Versionskontrolle (Git), virtuellen Umgebungen (venv) und Tests (pytest) bleibt das Projekt auch bei mehreren Teammitgliedern jederzeit nachvollziehbar und wartbar.

Mit diesen Erweiterungsoptionen bleibt das System langfristig attraktiv, ausbaufähig und zukunftssicher!
Alle vorgeschlagenen Features lassen sich durch die offene Architektur nachträglich und schrittweise integrieren.